

3. Semana 3. Métodos Numéricos Fundamentales

Objetivo de la cátedra

Aplicar técnicas de discretización y aproximación numérica para resolver problemas científicos y tecnológicos, comprendiendo los fundamentos de estabilidad, convergencia y condicionamiento que determinan la calidad de una solución computacional.

Temario

- Diferencias finitas: derivación numérica y orden de error.
- Integración numérica: cuadraturas clásicas y Gauss–Legendre.
- Estabilidad numérica y criterios de von Neumann.
- Convergencia y orden de convergencia.
- Condicionamiento numérico y número de condición.
- Introducción a ecuaciones diferenciales parciales (EDPs).
- Discretización en espacio y tiempo: esquemas explícitos e implícitos.

Propósito del tema. En las Semanas 1 y 2 se construyó la cadena conceptual fenómeno $\rightarrow$ modelo matemático y la infraestructura técnica para ejecutarla. Esta semana se establece la base matemática que hace que esa ejecución sea correcta: cómo pasar de una ecuación continua a un sistema discreto resoluble, qué le ocurre al error en ese proceso, cuándo un algoritmo falla catastróficamente aunque el código esté bien escrito, y cómo cuantificar la confianza en un resultado numérico. Sin estos criterios, una simulación es una caja negra; con ellos, es un argumento científico defendible.

Resultado de aprendizaje esperado

Al finalizar la sesión, el estudiante será capaz de: (i) derivar fórmulas de diferencias finitas de orden arbitrario a partir de la expansión de Taylor y verificar su orden de convergencia empíricamente; (ii) seleccionar e implementar cuadraturas numéricas con criterios de orden y costo para integrales de interés físico; (iii) determinar el criterio de estabilidad de un esquema en diferencias finitas mediante análisis de von Neumann y verificarlo en la práctica; (iv) calcular el número de condición de un sistema lineal e interpretar su efecto sobre el error de la solución; y (v) formular la discretización de una EDP parabólica simple y justificar la elección entre esquemas explícitos e implícitos.

3.1. Motivación: por qué los métodos numéricos importan en investigación doctoral

Toda simulación computacional requiere resolver, en algún punto, uno de los siguientes tres problemas fundamentales:

1. **Diferenciación:** estimar $\frac{df}{dx}$ o $\frac{\partial u}{\partial t}$ a partir de valores discretos de f o u .
2. **Integración:** evaluar $\int_a^b f(x) dx$ cuando f no tiene primitiva cerrada.
3. **Resolución de sistemas lineales:** dado $Ax = b$, encontrar x de forma estable y eficiente.

Estos tres problemas son el núcleo de prácticamente todos los proyectos doctorales del grupo. El análisis de deformaciones electro-mecánicas en un cable OPGW requiere integrar la ecuación de calor transitoria y resolver sistemas lineales de conductividad. La optimización de una bomba centrífuga mediante simulación CFD resuelve el sistema de Navier–Stokes discretizado, cuya parte difusiva genera sistemas lineales con matrices laplacianas. El análisis de losas y cimentaciones (Lerma Lomelí, Lerma Martínez) requiere resolver el sistema $Ku = f$ del método de elementos finitos, donde K es la matriz de rigidez global y su número de condición determina la calidad de la solución. El modelo de tráfico con agentes y UAV (García Ojeda) acumula integrales de densidad vehicular y derivadas de flujo. El sistema RAG (Rodríguez Morales) requiere álgebra lineal numérica para operaciones de similitud en espacios vectoriales de alta dimensión.

La conclusión es directa: entender estos métodos no es un trámite académico; es la condición técnica mínima para que cualquier simulación doctoral sea defendible ante un comité de revisión.

3.2. Diferencias finitas

3.2.1. Derivación a partir de la expansión de Taylor

La expansión de Taylor de f alrededor de x_0 es

$$f(x_0 + h) = f(x_0) + h f'(x_0) + \frac{h^2}{2} f''(x_0) + \frac{h^3}{6} f'''(x_0) + \mathcal{O}(h^4). \quad (91)$$

Despejando $f'(x_0)$:

$$f'(x_0) = \frac{f(x_0 + h) - f(x_0)}{h} - \frac{h}{2} f''(x_0) - \dots \quad (92)$$

El primer término es la *diferencia hacia adelante* (forward difference); el residuo $-\frac{h}{2}f''$ establece que el error de truncamiento es $\mathcal{O}(h)$. La *diferencia hacia atrás* se obtiene expandiendo $f(x_0 - h)$:

$$f'(x_0) = \frac{f(x_0) - f(x_0 - h)}{h} + \mathcal{O}(h). \quad (93)$$

La *diferencia centrada* se obtiene restando las expansiones de $f(x_0 + h)$ y $f(x_0 - h)$:

$$f'(x_0) = \frac{f(x_0 + h) - f(x_0 - h)}{2h} - \frac{h^2}{6} f'''(x_0) - \dots = \frac{f(x_0 + h) - f(x_0 - h)}{2h} + \mathcal{O}(h^2). \quad (94)$$

La diferencia centrada tiene un orden de convergencia superior (h^2 frente a h) porque los términos de error de orden impar se cancelan. Este es el principio general para construir fórmulas de mayor orden: combinar más puntos de evaluación de modo que los términos de error de orden bajo se anulen mutuamente.

3.2.2. Error de redondeo y paso óptimo

Existe un piso en el error que se debe al redondeo aritmético. Si $\varepsilon_{\text{mach}} \approx 2.22 \times 10^{-16}$ es el épsilon de máquina (IEEE 754 float64), el error total en la diferencia centrada es aproximadamente

$$E_{\text{total}}(h) \approx \underbrace{\frac{h^2}{6} |f'''(x_0)|}_{\text{truncamiento}} + \underbrace{\frac{2\varepsilon_{\text{mach}} |f(x_0)|}{h}}_{\text{redondeo}}. \quad (95)$$

Minimizando respecto a h :

$$h_{\text{opt}} = \left(\frac{3\varepsilon_{\text{mach}} |f|}{|f''|} \right)^{1/3} \approx \varepsilon_{\text{mach}}^{1/3} \approx 6 \times 10^{-6}. \quad (96)$$

Para la diferencia hacia adelante, el óptimo es $h_{\text{opt}} \approx \sqrt{\varepsilon_{\text{mach}}} \approx 1.5 \times 10^{-8}$. Estos valores no son configurables: son consecuencia directa de la aritmética del hardware. Usar h mucho más pequeño que h_{opt} *aumenta* el error por cancelación catastrófica.

3.2.3. Segunda derivada

La segunda derivada se obtiene combinando las expansiones de $f(x_0 + h)$ y $f(x_0 - h)$:

$$f''(x_0) = \frac{f(x_0 + h) - 2f(x_0) + f(x_0 - h))}{h^2} + \mathcal{O}(h^2). \quad (97)$$

Esta fórmula es el stencil de tres puntos del operador laplaciano. En 2D genera la ecuación de Poisson discreta; en espacio-tiempo genera el esquema explícito de la ecuación de calor. Es la fórmula más frecuente en simulación de campos físicos.

Tabla 12: Fórmulas de diferencias finitas más usadas y su orden de error de truncamiento.

Nombre	Fórmula	Orden
Primera derivada (adelante)	$\frac{f(x+h) - f(x)}{h}$	$\mathcal{O}(h)$
Primera derivada (atrás)	$\frac{f(x) - f(x-h)}{h}$	$\mathcal{O}(h)$
Primera derivada (centrada)	$\frac{f(x+h) - f(x-h)}{2h}$	$\mathcal{O}(h^2)$
Segunda derivada (centrada)	$\frac{f(x+h) - 2f(x) + f(x-h)}{h^2}$	$\mathcal{O}(h^2)$
Primera deriv. centrada $\mathcal{O}(4)$	$\frac{-f(x+2h) + 8f(x+h) - 8f(x-h) + f(x-2h)}{12h}$	$\mathcal{O}(h^4)$

3.2.4. Ejemplo 1 — Diferenciación numérica en el perfil de deformación de un cable OPGW

Contexto físico. El proyecto de Méndez Gutiérrez analiza las deformaciones electromecánicas del cable OPGW. Una de las cantidades críticas es la derivada del campo de deformación longitudinal $\varepsilon(x)$ respecto a la coordenada axial x , que es proporcional a la fuerza cortante en el cable. En la práctica, $\varepsilon(x)$ se obtiene de mediciones discretas con sensores de fibra óptica (FBG o BOTDR), por lo que la derivada debe calcularse numéricamente. El mismo problema aparece en el análisis de curvaturas de cables bajo arco eléctrico (Jurado Pérez).

Formulación matemática. Se utiliza como función de prueba verificable un perfil sintético con derivada analítica conocida:

$$f(x) = \sin(2\pi x) e^{-0.5x}, \quad f'(x) = (2\pi \cos(2\pi x) - 0.5 \sin(2\pi x)) e^{-0.5x}. \quad (98)$$

El error absoluto de cada esquema se evalúa en $x_0 = 0.75$ para un barrido de $h \in [10^{-7}, 10^{-0.5}]$. En la [Figura 20](#) se muestra la interpretación geométrica y la curva de convergencia.

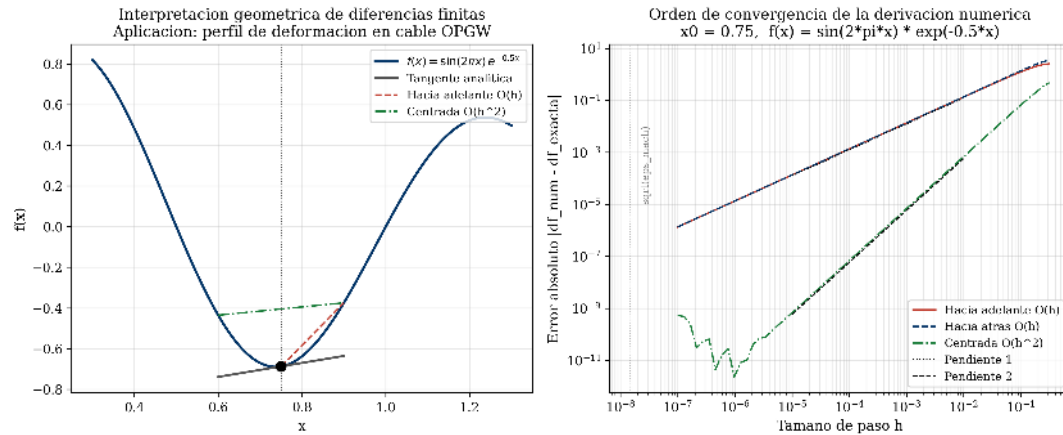


Figura 20: Izquierda: interpretación geométrica de las fórmulas de diferencias finitas sobre el perfil de deformación $f(x) = \sin(2\pi x)e^{-0.5x}$. Derecha: error absoluto de derivación numérica en función de h en escala log-log. Las pendientes confirman $O(h)$ para diferencias no centradas y $O(h^2)$ para la centrada. El piso de error a la izquierda del mínimo se debe a cancelación catastrófica por aritmética de punto flotante (Ec. 95).

Interpretación cuantitativa.

- Para $h \approx 10^{-2}$: error de diferencia centrada $\sim 10^{-4}$; error de diferencia hacia adelante $\sim 10^{-2}$. La centrada es $\sim 100\times$ más precisa con el mismo costo computacional.
- El mínimo de error de la diferencia centrada ocurre alrededor de $h \approx 10^{-5}$, consistente con $h_{\text{opt}} \approx \varepsilon_{\text{mach}}^{1/3}$.
- A la izquierda del mínimo, la pendiente se invierte: reducir h empeora el resultado. Este es el régimen de error dominado por redondeo, no por truncamiento.

Limitaciones. Este análisis asume que f es evaluable con precisión de máquina. En datos experimentales (medición con sensor BOTDR), el piso de error no es $\varepsilon_{\text{mach}}$ sino el ruido del instrumento $\sigma_{\text{instrumento}}$. En ese caso, $h_{\text{opt}} \sim (\sigma_{\text{instrumento}}/|f'''|)^{1/3}$ y es considerablemente mayor. Usar diferencias finitas con h demasiado pequeño sobre datos experimentales amplifica el ruido; se requiere regularización o filtrado previo.

```

1      """
2      Semana 3 - Ejemplo 1: Diferenciación numérica y
orden de convergencia
3      Aplicación: perfil de deformación en cable OPGW (
Mendez Gutierrez)
4      """
5      import numpy as np
6      import matplotlib.pyplot as plt
7
    
```

```

8      # ---- Funcion de prueba y derivada analitica
9      -----
10     def f(x):
11         return np.sin(2.0 * np.pi * x) * np.exp(-0.5 * x)
12
13     def df_analitica(x):
14         return (2.0*np.pi*np.cos(2.0*np.pi*x)
15             - 0.5*np.sin(2.0*np.pi*x)) * np.exp(-0.5*x)
16
17     x0      = 0.75
18     df_exact = df_analitica(x0)
19
20     # ---- Barrido de pasos h
21     -----
22     h_vals  = np.logspace(-7, -0.5, 60)
23     err_fwd = np.abs((f(x0 + h_vals) - f(x0))
24 / h_vals    - df_exact)
25     err_bwd = np.abs((f(x0)
26 / h_vals    - df_exact) - f(x0 - h_vals))
27     err_ctr = np.abs((f(x0 + h_vals) - f(x0 - h_vals)) /
28 (2.0*h_vals) - df_exact)
29
30     # ---- Reporte de valores selectos
31     -----
32     for h in [1e-2, 1e-4, 1e-6]:
33         df_c = (f(x0+h) - f(x0-h)) / (2*h)
34         print(f"h={h:.0e}: df_centrada={df_c:.8f}, "
35             f"df_exacta={df_exact:.8f}, "
36             f"error={abs(df_c - df_exact):.2e}")
37
38     # ---- Verificacion del orden: pendiente en log-log
39     -----
40     # Para dos puntos h1, h2: pendiente = log(err2/err1)
41     / log(h2/h1)
42     h1, h2 = 1e-4, 1e-3
43     e1c = abs((f(x0+h1)-f(x0-h1))/(2*h1) - df_exact)
44     e2c = abs((f(x0+h2)-f(x0-h2))/(2*h2) - df_exact)
45     orden_centrada = np.log(e2c/e1c) / np.log(h2/h1)
46     print(f"Orden verificado (centrada): {orden_centrada
47 :.3f} (esperado: 2.0)")
48
49     # ---- Figura convergencia

```

```

41         fig, ax = plt.subplots(figsize=(7, 5))
42         ax.loglog(h_vals, err_fwd, label='Hacia adelante O(h
)')
43         ax.loglog(h_vals, err_bwd, '--', label='Hacia atras
O(h)')
44         ax.loglog(h_vals, err_ctr, '-.', label='Centrada O(h
^2)')
45         ax.set_xlabel('Paso h'); ax.set_ylabel('|error|')
46         ax.set_title('Convergencia de la derivacion numerica
')
47         ax.legend(); ax.grid(alpha=0.3, which='both')
48         plt.tight_layout()
49         plt.savefig('fig_s3_diferencias_finitas.png', dpi
=200)
50         plt.show()
51

```

Listing 19: Semana 3 – Ejemplo 1: Diferenciación numérica con verificación de orden. Ejecutar en Google Colab.

3.3. Integración numérica

3.3.1. Reglas de cuadratura clásicas

Sea $I = \int_a^b f(x) dx$. La estrategia general es particionar $[a, b]$ en N subintervalos de tamaño $h = (b - a)/N$ y aproximar f sobre cada subintervalo mediante un polinomio de grado p . El error de truncamiento global es $\mathcal{O}(h^{p+1})$ para $f \in C^{p+1}[a, b]$.

Las reglas más utilizadas se resumen en la [Tabla 13](#). El punto crítico es que el orden de la regla determina el número de evaluaciones de f necesarias para alcanzar una tolerancia dada. Simpson gasta $N + 1$ evaluaciones pero alcanza $\mathcal{O}(h^4)$, lo que lo hace 10^4 veces más preciso que la regla de rectángulos para el mismo h .

Tabla 13: Comparación de reglas de cuadratura numérica. $h = (b - a)/N$.

Método	Fórmula	Orden	Restricción
Rectángulos (izq.)	$h \sum_{i=0}^{N-1} f(x_i)$	$\mathcal{O}(h)$	Ninguna
Trapecios	$h \left[\frac{f(a)+f(b)}{2} + \sum_{i=1}^{N-1} f(x_i) \right]$	$\mathcal{O}(h^2)$	Ninguna
Simpson $\frac{1}{3}$	$\frac{h}{3} [f_0 + 4f_1 + 2f_2 + \dots + f_N]$	$\mathcal{O}(h^4)$	N par
Gauss–Legendre (n puntos)	$\frac{b-a}{2} \sum_{k=1}^n w_k f(x_k^*)$	$\mathcal{O}(h^{2n})$	Puntos x_k^* fijos

3.3.2. Cuadratura de Gauss–Legendre

La cuadratura de Gauss–Legendre selecciona los n puntos $\{x_k^*\}$ y pesos $\{w_k\}$ en $[-1, 1]$ de manera que se integra exactamente todo polinomio de grado $\leq 2n - 1$. Para una función suave, con n puntos se logra un orden efectivo de $\mathcal{O}(h^{2n})$, lo cual es exponencialmente superior a las cuadraturas de Newton-Cotes para el mismo número de evaluaciones. La transformación al intervalo $[a, b]$ es

$$\int_a^b f(x) \, dx = \frac{b-a}{2} \sum_{k=1}^n w_k f\left(\frac{b-a}{2} x_k^* + \frac{a+b}{2}\right). \quad (99)$$

En `scipy`, los puntos y pesos se obtienen con `numpy.polynomial.legendre.leggauss(n)`.

3.3.3. Ejemplo 2 — Energía disipada por arco eléctrico en cortocircuito

Contexto físico. El proyecto de Jurado Pérez analiza el arco eléctrico que incide sobre un cable OPGW durante un cortocircuito. Una cantidad fundamental es la energía total disipada por el arco:

$$E = \int_0^T P(t) \, dt, \quad P(t) = I^2 R(T(t)), \quad (100)$$

donde $R(T)$ es la resistencia dependiente de la temperatura. El modelo de Cassie simplificado supone una resistencia de la columna del arco que decae exponencialmente:

$$P(t) = I^2 R_0 e^{-\alpha t}, \quad (101)$$

con integral exacta analítica disponible:

$$E_{\text{exacta}} = \frac{I^2 R_0}{\alpha} (1 - e^{-\alpha T}). \quad (102)$$

Los parámetros $I = 10 \text{ kA}$, $R_0 = 50 \text{ m}\Omega$, $\alpha = 1500 \text{ s}^{-1}$, $T = 5 \text{ ms}$ dan $E_{\text{exacta}} \approx 3331.5 \text{ J}$. Este valor es la referencia contra la que se mide el error de cada método numérico.

La [Figura 21](#) muestra la integración sobre la curva de potencia y la convergencia de cada método. Nótese que Gauss–Legendre con un número modesto de puntos supera en precisión a Simpson con $N = 1000$ subintervalos.

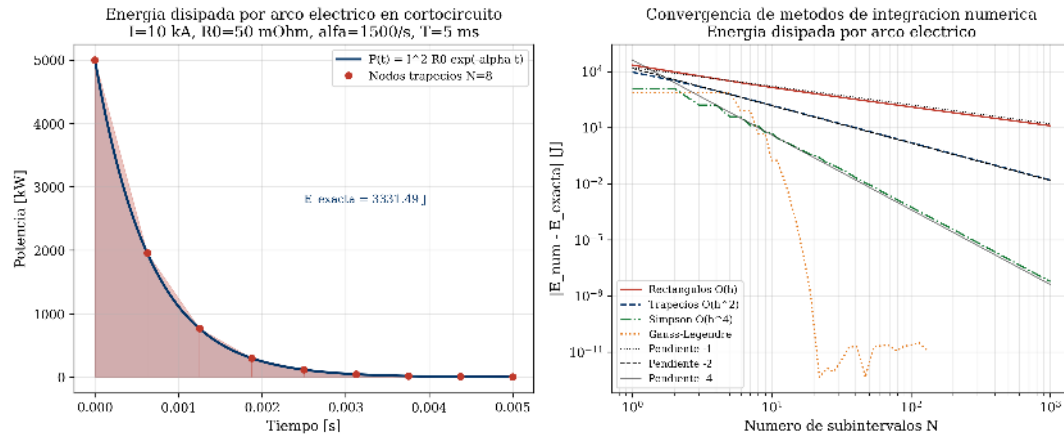


Figura 21: Izquierda: potencia disipada por arco eléctrico $P(t) = I^2 R_0 e^{-\alpha t}$ e ilustración de la regla de los trapecios con $N = 8$ subintervalos. Derecha: convergencia del error absoluto $|E_{\text{num}} - E_{\text{exacta}}|$ en función del número de subintervalos N para cuatro métodos. Las pendientes empíricas confirman los órdenes teóricos: $\mathcal{O}(h)$, $\mathcal{O}(h^2)$, $\mathcal{O}(h^4)$ para rectángulos, trapecios y Simpson respectivamente.

```

1      """
2      Semana 3 - Ejemplo 2: Integracion numerica - energia
      por arco electrico
3      Modelo Cassie simplificado. Aplicacion: proyecto
      Jurado Perez (OPGW arco).
4      """
5      import numpy as np
6      from scipy import integrate
7      from numpy.polynomial.legendre import leggauss
8      import matplotlib.pyplot as plt
9
10     # ---- Parametros fisicos del arco
11     -----
12     I      = 10e3          # corriente de cortocircuito [A]
13     R0     = 50e-3         # resistencia inicial del arco [
      Ohm]
14     alpha  = 1.5e3         # tasa de extincion [1/s] -
      parametro de Cassie
15     T_fin  = 5e-3         # duracion del evento [s]
16
17     def P(t):
18         """Potencia disipada:  $P(t) = I^2 * R_0 * \exp(-\alpha * t)$ 
19         ) [W] """
20         return I**2 * R0 * np.exp(-alpha * t)

```

```

20      # ---- Integral exacta (referencia analitica)
      -----
21      E_exacta = I**2 * R0 / alpha * (1.0 - np.exp(-alpha
      * T_fin))
22      print(f"Energia exacta: {E_exacta:.4f} J")
23
24      # ---- Implementaciones de los metodos de cuadratura
      -----
25      def trapecios(f, a, b, N):
26          x = np.linspace(a, b, N+1)
27          return np.trapezoid(f(x), x)
28
29      def simpson(f, a, b, N):
30          if N % 2 != 0: N += 1
31          x = np.linspace(a, b, N+1)
32          return integrate.simpson(f(x), x=x)
33
34      def gauss_legendre(f, a, b, n):
35          xi, wi = leggauss(n)
36          t_pts = 0.5*(b - a)*xi + 0.5*(b + a)
37          return 0.5*(b - a) * float(np.dot(wi, f(t_pts)))
38
39      # ---- Convergencia de los metodos
      -----
40      N_vals = np.unique(np.logspace(0.3, 3.0, 50).astype(
      int))
41      err_trap = [abs(trapecios(P, 0, T_fin, N) - E_exacta
      ) for N in N_vals]
42      err_simp = [abs(simpson(P, 0, T_fin, N) -
      E_exacta) for N in N_vals]
43      err_gl = [abs(gauss_legendre(P, 0, T_fin, max(2,N
      //2)) - E_exacta)
44      for N in N_vals]
45
46      # ---- Verificacion del orden (trapecios, dos puntos
      ) -----
47      for N1, N2 in [(10, 100), (100, 1000)]:
48          e1 = abs(trapecios(P, 0, T_fin, N1) - E_exacta)
49          e2 = abs(trapecios(P, 0, T_fin, N2) - E_exacta)
50          orden = np.log(e1/e2) / np.log(N2/N1)
51          print(f"Orden trapecios N={N1}..{N2}: {orden:.3f} (
      esperado: 2.0)")
    
```

```

52
53      # ---- Comparacion puntual a N=100
54      -----
55      for nombre, metodo, args in [
56          ('Trapecios', trapecios, (P, 0, T_fin, 100)),
57          ('Simpson', simpson, (P, 0, T_fin, 100)),
58          ('Gauss-Leg', gauss_legendre, (P, 0, T_fin, 50)),
59      ]:
60          E_num = metodo(*args)
61          print(f"{nombre:12s}: E = {E_num:.6f} J, error = {
62      abs(E_num-E_exacta):.2e} J")
63
64      # ---- Figura
65      -----
66
67      fig, ax = plt.subplots(figsize=(7, 5))
68      ax.loglog(N_vals, err_trap, label='Trapecios 0(h^2)'
69      )
70      ax.loglog(N_vals, err_simp, '--', label='Simpson 0(h
71      ^4)')
72      ax.loglog(N_vals, err_gl, '-.', label='Gauss-
73      Legendre')
74      ax.set_xlabel('N subintervalos / puntos de
75      cuadratura')
76      ax.set_ylabel('|E_num - E_exacta| [J]')
77      ax.set_title('Convergencia: energia disipada por
78      arco electrico')
79      ax.legend(); ax.grid(alpha=0.3, which='both')
80      plt.tight_layout(); plt.savefig('fig_s3_integracion.
81      png', dpi=200)
82      plt.show()

```

Listing 20: Semana 3 – Ejemplo 2: Integración numérica de la energía disipada por arco. Ejecutar en Google Colab.

3.4. Estabilidad numérica

3.4.1. Ecuaciones diferenciales ordinarias: esquemas explícitos e implícitos

Considérese la EDO lineal de prueba $\frac{dy}{dt} = \lambda y$, con $\text{Re}(\lambda) < 0$ (sistema estable). El método de Euler explícito produce la recurrencia

$$y^{n+1} = (1 + \Delta t \lambda) y^n, \quad (103)$$

que converge solo si $|1 + \Delta t \lambda| \leq 1$, es decir, $\Delta t \leq 2/|\lambda|$. Este es el *criterio de estabilidad de Euler explícito*. Para sistemas físicos rígidos (*stiff*), $|\lambda|$ es grande y el criterio impone pasos de tiempo extremadamente pequeños, haciendo el cálculo prohibitivo.

El método de Euler implícito produce

$$y^{n+1} = \frac{y^n}{1 - \Delta t \lambda}, \quad (104)$$

cuyo factor de amplificación $|1/(1 - \Delta t \lambda)| \leq 1$ para todo $\Delta t > 0$ cuando $\text{Re}(\lambda) < 0$. El método es *incondicionalmente estable* (*A-estable*), pero requiere resolver un sistema (no lineal en general) en cada paso.

El método de Crank–Nicolson (media aritmética de Euler explícito e implícito) es también *A-estable* y tiene orden $\mathcal{O}(\Delta t^2)$:

$$y^{n+1} = \frac{1 + \frac{1}{2}\Delta t \lambda}{1 - \frac{1}{2}\Delta t \lambda} y^n. \quad (105)$$

3.4.2. Criterio de estabilidad CFL para EDPs

Para la ecuación de difusión 1D $\frac{\partial u}{\partial t} = \alpha \frac{\partial^2 u}{\partial x^2}$ discretizada con Euler explícito en tiempo y diferencias centradas en espacio:

$$u_i^{n+1} = u_i^n + \frac{\alpha \Delta t}{\Delta x^2} (u_{i+1}^n - 2u_i^n + u_{i-1}^n), \quad (106)$$

el análisis de von Neumann establece que el número de Fourier $r = \alpha \Delta t / \Delta x^2$ debe satisfacer

$$r = \frac{\alpha \Delta t}{\Delta x^2} \leq \frac{1}{2}. \quad (107)$$

Este criterio impone una relación cuadrática entre el paso temporal y el paso espacial: reducir Δx a la mitad obliga a reducir Δt en un factor de 4. Para Δx pequeño, esto hace al esquema explícito impráctico. Los esquemas implícitos (Euler implícito, Crank–Nicolson, θ -method con $\theta \geq 0.5$) eliminan esta restricción.

3.4.3. Ejemplo 3 — Dinámica térmica de cable OPGW: estabilidad de esquemas numéricos

Contexto físico. La temperatura de un cable OPGW durante un cortocircuito determina si las fibras ópticas sufren daño permanente. Se formula el modelo concentrado (*lumped capacitance*):

$$\rho c_p V \frac{dT}{dt} = I^2 R(T) - h_{\text{conv}} A_s (T - T_\infty), \quad (108)$$

donde $R(T) = R_{20} [1 + \alpha_R(T - 20)]$ es la resistencia dependiente de temperatura (modelo lineal de IEC 61597). El eigenvalor dominante del sistema linealizado en $T_0 = 20^\circ\text{C}$ es

$$\lambda \approx \frac{I^2 R_{20} \alpha_R - h_{\text{conv}} A_s}{\rho c_p V}, \quad (109)$$

que determina el criterio de estabilidad $\Delta t_{\text{crit}} = 2/|\lambda|$.

La Figura 22 muestra los cuatro esquemas integrados con el mismo paso de tiempo ($\Delta t = 2.5 \Delta t_{\text{crit}}$): el Euler explícito diverge, mientras que el Euler implícito y Crank–Nicolson convergen correctamente. El panel derecho verifica empíricamente los órdenes $\mathcal{O}(\Delta t)$ y $\mathcal{O}(\Delta t^2)$ respectivamente.

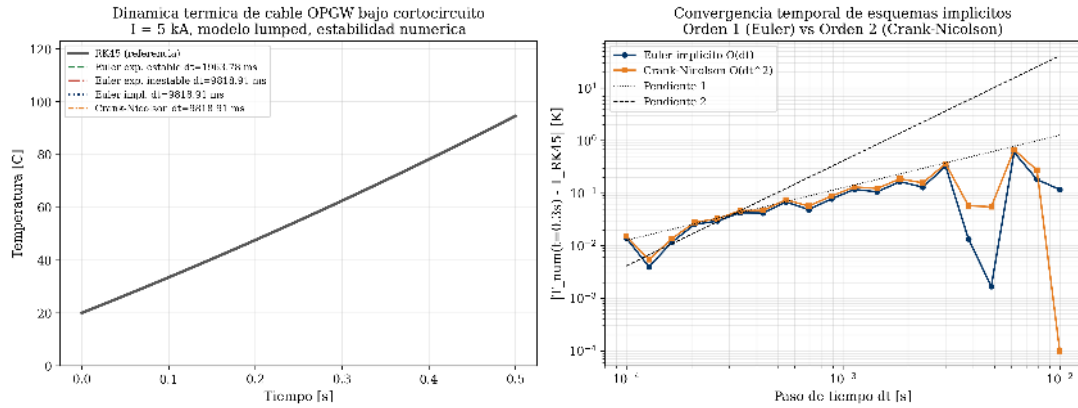


Figura 22: Izquierda: temperatura del cable OPGW bajo cortocircuito integrada con cuatro esquemas. El Euler explícito con $\Delta t = 2.5 \Delta t_{\text{crit}}$ diverge, mientras que el Euler implícito y Crank–Nicolson convergen. La solución de referencia es RK45 de `scipy.solve_ivp`. Derecha: error al tiempo $t = 0.3\text{ s}$ en función de Δt , verificando órdenes $\mathcal{O}(\Delta t)$ y $\mathcal{O}(\Delta t^2)$.

```

1      """
2      Semana 3 - Ejemplo 3: Estabilidad numerica en ODE
3      rigida
4      Problema fisico: calentamiento de cable OPGW en
5      cortocircuito
6      Modelo: rho*cp*V * dT/dt = I^2*R(T) - h*A*(T - T_inf)
7
8      """
9
10     import numpy as np
11     import matplotlib.pyplot as plt
12     from scipy.integrate import solve_ivp
13
14     # ---- Parametros fisicos (aluminio OPGW)
15     -----
16     rho      = 2700.0          # densidad [kg/m^3]

```

```

12         cp      = 900.0          # calor especifico [J/(kg·K)
13     ]
14         r_cab   = 5.0e-3          # radio del cable [m]
15         L       = 1.0            # longitud unitaria [m]
16         V       = np.pi * r_cab**2 * L
17         A_s     = 2.0 * np.pi * r_cab * L
18         R20     = 1.0e-3          # resistencia a 20 C [Ohm]
19         alpha_R = 3.9e-3          # coef. termico de
    resistencia [1/K]
20         h_conv  = 10.0            # coef. conveccion [W/(m^2·K
21     )]
22         T_inf   = 20.0            # temperatura ambiente [C]
23         T0      = 20.0            # condicion inicial [C]
24         I_sc    = 5e3              # corriente de cortocircuito
25     [A]
26         t_end   = 0.5             # duracion [s]
27         mC      = rho * cp * V    # capacidad termica [J/K]
28
29     def R_T(T):
30         return R20 * (1.0 + alpha_R * (T - 20.0))
31
32     def dTdt(t, T_arr):
33         T = T_arr[0]
34         return [(I_sc**2 * R_T(T) - h_conv * A_s * (T -
35     T_inf)) / mC]
36
37     # ---- Eigenvalor linealizado y paso critico
38     -----
39     lam = (I_sc**2 * R20 * alpha_R - h_conv * A_s) / mC
40     dt_crit = 2.0 / abs(lam)
41     print(f"Lambda      = {lam:.4f} /s")
42     print(f"dt_critico = {dt_crit*1e3:.4f} ms")
43
44     # ---- Referencia con RK45 (alta precision)
45     -----
46     sol_ref = solve_ivp(dTdt, [0, t_end], [T0], method='
47     RK45',
48     dense_output=True, rtol=1e-9, atol=1e-12)
49     t_ref = np.linspace(0, t_end, 2000)
50     T_ref = sol_ref.sol(t_ref)[0]
51     print(f"T_max (RK45) = {T_ref.max():.2f} C")

```

```

46      # ---- Euler explícito
47      -----
48      def euler_exp(dt, t_fin):
49          N = int(t_fin / dt); t = np.linspace(0, N*dt, N+1)
50          T = np.zeros(N+1); T[0] = T0
51          for k in range(N):
52              T[k+1] = T[k] + dt * dTdt(t[k], [T[k]])[0]
53          return t, T
54
55      # ---- Euler implícito con Newton
56      -----
57      def euler_imp(dt, t_fin):
58          N = int(t_fin / dt); t = np.linspace(0, N*dt, N+1)
59          T = np.zeros(N+1); T[0] = T0
60          dfdt_lin = (I_sc**2*R20*alpha_R - h_conv*A_s) / mC
61          # Jacobiano linealizado
62          for k in range(N):
63              Tk = T[k]
64              for _ in range(20): # Newton
65                  fval = dTdt(t[k+1], [Tk])[0]
66                  F = Tk - T[k] - dt*fval
67                  dF = 1.0 - dt*dfdt_lin
68                  Tk -= F/dF
69                  if abs(F) < 1e-10: break
70              T[k+1] = Tk
71          return t, T
72
73      # ---- Crank-Nicolson
74      -----
75      def crank_nicolson(dt, t_fin):
76          N = int(t_fin / dt); t = np.linspace(0, N*dt, N+1)
77          T = np.zeros(N+1); T[0] = T0
78          dfdt_lin = (I_sc**2*R20*alpha_R - h_conv*A_s) / mC
79          for k in range(N):
80              fn = dTdt(t[k], [T[k]])[0]
81              Tk = T[k] + dt*fn
82              for _ in range(20):
83                  fn1 = dTdt(t[k+1], [Tk])[0]
84                  F = Tk - T[k] - 0.5*dt*(fn + fn1)
85                  dF = 1.0 - 0.5*dt*dfdt_lin
86                  Tk -= F/dF
87                  if abs(F) < 1e-10: break

```

```

84         T[k+1] = Tk
85         return t, T
86
87         # ---- Integracion con dt > dt_crit (Euler exp debe
divergir) -----
88         dt_inestable = 2.5 * dt_crit
89         dt_estable   = 0.5 * dt_crit
90
91         t_es, T_es = euler_exp(dt_estable, min(t_end,
0.15))
92         t_ei_d, T_ei_d = euler_exp(dt_inestable, min(t_end,
0.05)) # solo primeros pasos
93         t_ei, T_ei = euler_imp(dt_inestable, t_end)
94         t_cn, T_cn = crank_nicolson(dt_inestable, t_end)
95
96         print(f"\nT final RK45      = {T_ref[-1]:.4f} C")
97         print(f"T final Euler imp = {T_ei[-1]:.4f} C")
98         print(f"T final Crank-Nic = {T_cn[-1]:.4f} C")
99
100        # ---- Figura
-----
101        fig, axes = plt.subplots(1, 2, figsize=(13, 5))
102        ax = axes[0]
103        ax.plot(t_ref, T_ref, lw=2.5, color='gray', label='
RK45 (referencia)')
104        ax.plot(t_es, T_es, lw=1.5, ls='--', color='green'
,
105        label=f'Euler exp. estable dt={dt_estable*1e3:.1f}
ms')
106        ax.plot(t_ei_d, T_ei_d, lw=1.5, ls='-.', color='red'
,
107        label=f'Euler exp. INESTABLE dt={dt_inestable*1e3:.1
f} ms')
108        ax.plot(t_ei, T_ei, lw=1.5, ls=':', color='blue',
109        label=f'Euler impl. dt={dt_inestable*1e3:.1f} ms')
110        ax.plot(t_cn, T_cn, lw=1.5, color='orange',
111        label=f'Crank-Nicolson dt={dt_inestable*1e3:.1f} ms'
)
112        ax.set_xlabel('Tiempo [s]'); ax.set_ylabel('
Temperatura [C]')
113        ax.set_title('Temperatura de cable OPGW bajo
cortocircuito\n')

```



```

114         'Comparacion de esquemas: estabilidad')
115         ax.legend(fontsize=8); ax.grid(alpha=0.3)
116         ax.set_ylim([T0 - 10, T_ref.max()*1.2])
117
118         # Convergencia del orden
119         ax = axes[1]
120         dt_arr = np.logspace(-4, -2, 20)
121         T_ref_check = float(sol_ref.sol(0.3)[0])
122         err_ei = [abs(euler_imp(dt, 0.3)[1][-1] -
123 T_ref_check) for dt in dt_arr]
124         err_cn = [abs(crank_nicolson(dt, 0.3)[1][-1] -
125 T_ref_check) for dt in dt_arr]
126         ax.loglog(dt_arr, err_ei, 'o-', label='Euler
127 implícito 0(dt)')
128         ax.loglog(dt_arr, err_cn, 's--', label='Crank-
129 Nicolson 0(dt^2)')
130         idx = 5
131         ax.loglog(dt_arr, err_ei[idx]*(dt_arr/dt_arr[idx]),
132 'k:', label='Pendiente 1')
133         ax.loglog(dt_arr, err_cn[idx]*(dt_arr/dt_arr[idx])
134 **2, 'k--', label='Pendiente 2')
135         ax.set_xlabel('Paso de tiempo dt [s]')
136         ax.set_ylabel('|T_num - T_RK45| [K]')
137         ax.set_title('Orden de convergencia temporal (t=0.3
138 s)')
139
140         ax.legend(); ax.grid(alpha=0.3, which='both')
141
142         plt.tight_layout()
143         plt.savefig('fig_s3_estabilidad_ode.png', dpi=200)
144         plt.show()
145
146
147

```

Listing 21: Semana 3 – Ejemplo 3: Estabilidad de esquemas para la ODE térmica del cable OPGW. Ejecutar en Google Colab.

3.5. Condicionamiento numérico

3.5.1. Número de condición y sensibilidad de la solución

Sea $Ax = b$ un sistema lineal. Si b tiene una perturbación δb , la perturbación relativa en la solución satisface

$$\frac{\|\delta \mathbf{x}\|}{\|\mathbf{x}\|} \leq \kappa(A) \frac{\|\delta \mathbf{b}\|}{\|\mathbf{b}\|}, \quad (110)$$

donde el *número de condición* es $\kappa(A) = \|A\| \|A^{-1}\| = \sigma_{\max}/\sigma_{\min}$ (razón entre el valor singular máximo y mínimo). La [Ecuación \(110\)](#) establece que un error relativo del 1 % en $\mathbf{b}$ puede producir un error relativo de hasta $\kappa(A)$ % en $\mathbf{x}$. Si $\kappa(A) \approx 10^{12}$, la solución puede perder 12 dígitos significativos.

Fuentes de mal condicionamiento en investigación doctoral.

- **Matrices de rigidez FEM con geometrías muy distorsionadas:** elementos triangulares con ángulos cercanos a 0° o 180° producen $\kappa(K) \gg 1$.
- **Ajuste de curvas con funciones casi colineales:** la matriz de diseño $\Phi^\top \Phi$ es mal condicionada si las funciones base son casi linealmente dependientes.
- **Sistemas de ecuaciones de cimentaciones sobredeterminados:** cuando se combinan ecuaciones de equilibrio con restricciones de compatibilidad que involucran diferentes órdenes de magnitud (Lerma Lomelí, Lerma Martínez).
- **Correlaciones de señales en sistemas RAG:** las matrices de similitud entre documentos pueden tener espectros muy degenerados (Rodríguez Morales).

3.5.2. Relación con el espectro de la matriz

Para matrices simétricas definidas positivas (SPD), como las matrices de rigidez FEM y las matrices laplacianas de diferencias finitas:

$$\kappa_2(A) = \frac{\lambda_{\max}}{\lambda_{\min}}, \quad (111)$$

donde $\lambda_{\max}$ y $\lambda_{\min}$ son los eigenvalores extremos. Para la matriz laplaciana 1D de $N \times N$:

$$\lambda_k = -4 \sin^2 \left(\frac{k\pi}{2(N+1)} \right), \quad k = 1, \dots, N, \quad (112)$$

lo que da $\kappa_2 \sim N^2$. Esto tiene una consecuencia directa: el gradiente conjugado (CG) converge en $\mathcal{O}(\sqrt{\kappa})$ iteraciones, es decir, en $\mathcal{O}(N)$ iteraciones para la laplaciana. Un preconditionador que reduzca κ de N^2 a $\mathcal{O}(1)$ reduce el costo de iteraciones de $\mathcal{O}(N)$ a $\mathcal{O}(1)$ por problema lineal.

3.5.3. Ejemplo 4 — Condicionamiento en sistemas de ecuaciones de ingeniería

Contexto físico. Los proyectos de análisis estructural (Lerma Lomelí, Lerma Martínez) resuelven sistemas lineales $K\mathbf{u} = \mathbf{f}$ provenientes de la discretización de losas y cimentaciones. El condicionamiento de K depende del refinamiento de la malla y de la

relación de aspecto de los elementos. El mismo problema aparece en la calibración de sensores (Álvarez Sánchez) y en sistemas de ecuaciones en diseño de bombas (Leyva Jiménez).

La Figura 23 muestra cuatro aspectos del condicionamiento: el crecimiento exponencial de κ para la matriz de Hilbert (caso canónico de mal condicionamiento), la ampliación del error de solución proporcional a κ , el espectro de la matriz laplaciana 1D, y el costo computacional de LU directo frente a gradiente conjugado en función del tamaño del sistema.

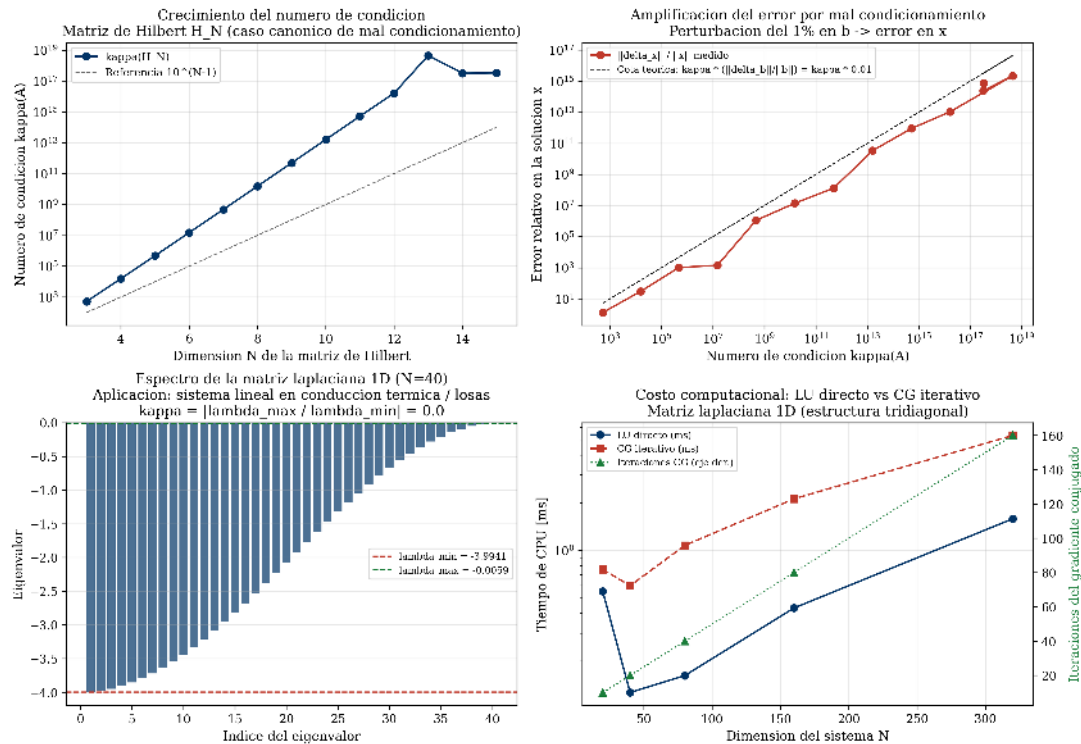


Figura 23: Panel A: crecimiento del número de condición $\kappa(H_N)$ para la matriz de Hilbert de dimensión N . Panel B: error relativo en la solución x ante una perturbación del 1 % en b , proporcional a $\kappa(A)$ conforme a la cota (Ec. 110). Panel C: espectro de eigenvalores de la matriz laplaciana 1D ($N = 40$); la relación $\kappa = |\lambda_{\max}/\lambda_{\min}|$ controla la velocidad de convergencia de métodos iterativos. Panel D: tiempo de CPU (ms) para LU directo vs gradiente conjugado (CG) en función del tamaño del sistema.

```

1      """
2      Semana 3 - Ejemplo 4: Condicionamiento numerico
3      Aplicacion: sistemas lineales en analisis
4      estructural (FEM),
5      calibracion de sensores y modelos de optimizacion.
6      """
7      import numpy as np
8      from scipy import linalg
    
```

```

8         from scipy.sparse import diags
9         from scipy.sparse.linalg import cg as sparse_cg
10        import matplotlib.pyplot as plt
11        import time
12
13        #
14        =====
15
16        # PARTE 1: Numero de condicion de la matriz de
17        Hilbert
18        #
19        =====
20
21        N_vals = range(3, 16)
22        kappas = [np.linalg.cond(linalg.hilbert(N)) for N in
23        N_vals]
24        for N, k in zip(N_vals, kappas):
25            n_digits = int(np.floor(np.log10(k))) if k > 1 else
26            0
27            print(f"N={N:2d}: kappa = {k:.2e} (~{n_digits}
28            digitos perdidos)")
29
30        #
31        =====
32
33        # PARTE 2: Efecto del condicionamiento en la
34        solucion
35        #
36        =====
37
38        np.random.seed(2024)
39        print("\nPerturbacion 1% en b -> error en x:")
40        for N in [5, 8, 10, 12]:
41            A = linalg.hilbert(N)
42            x_true = np.ones(N)
43            b = A @ x_true
44            db = 0.01 * b * (2*np.random.rand(N) - 1) # 1% de
45            perturbacion en b
46            x_pert = np.linalg.solve(A, b + db)
47            err_rel = np.linalg.norm(x_pert - x_true) / np.
48            linalg.norm(x_true)
49            k = np.linalg.cond(A)

```

```

35         print(f"N={N}: kappa={k:.2e}, ||dx||/||x||={err_rel
:.2e}, "
36         f"cota_teorica={k*0.01:.2e}")
37
38         #
=====
39         # PARTE 3: Espectro de la matrix laplaciana 1D
40         #
=====
41
42         N_lap = 40
43         dm = -2.0 * np.ones(N_lap)
44         do = 1.0 * np.ones(N_lap - 1)
45         A_lap = np.diag(dm) + np.diag(do, 1) + np.diag(do,
-1)
46
47         eigvals = np.sort(np.linalg.eigvalsh(A_lap))
48
49         # Eigenvalores analiticos para verificacion
50         k_vals = np.arange(1, N_lap+1)
51         lam_analiticos = -4.0 * np.sin(k_vals*np.pi / (2*(
N_lap+1)))*2
52         print(f"\nEigenvalor min (numerico): {eigvals[0]:.6f
}")
53
54         print(f"Eigenvalor min (analitico): {lam_analiticos.
min():.6f}")
55         print(f"Error relativo: {abs(eigvals[0]-
lam_analiticos.min())/abs(lam_analiticos.min()):.2e}")
56
57         kappa_lap = abs(eigvals[-1] / eigvals[0])
58         print(f"kappa (Laplaciana 1D, N={N_lap}): {kappa_lap
:.4f}")
59         print(f"Prediccion teorica  $N^2 * (\pi/2)^2 \sim \{N\_lap
**2 * (np.pi/2)**2 / (N\_lap+1)**2:.4f\}$ ")
60
61         #
=====
62
63         # PARTE 4: LU vs CG - comparacion de costo
64         #
=====

```

```

61         print("\nComparacion LU vs CG (Laplaciana 1D):")
62         print(f"{'N':>6}   {'t_LU [ms]':>10}   {'t_CG [ms'
        ]':>10}   {'iter_CG':>8}")
63         for N in [20, 50, 100, 200, 500]:
64             dm_s = -2.0*np.ones(N); do_s = np.ones(N-1)
65             A_s = diags([do_s, dm_s, do_s], [-1, 0, 1], format=
        'csc')
66             A_d = A_s.toarray()
67             b = np.ones(N)
68
69             t0 = time.perf_counter()
70             x_lu = linalg.solve(A_d, b)
71             t_lu = (time.perf_counter() - t0)*1e3
72
73             iters = [0]
74             def cb(xk): iters[0] += 1
75
76             t0 = time.perf_counter()
77             x_cg, info = sparse_cg(A_s, b, callback=cb, rtol=1e
        -10)
78             t_cg = (time.perf_counter() - t0)*1e3
79
80             err = np.linalg.norm(x_lu - x_cg) / np.linalg.norm(
        x_lu)
81             print(f"{'N':6d}   {'t_lu':10.3f}   {'t_cg':10.3f}   {'iters
        [0]:8d}   err={err:.2e}")
82
    
```

Listing 22: Semana 3 – Ejemplo 4: Condicionamiento numérico y resolución de sistemas lineales. Ejecutar en Google Colab.

3.6. Introducción a ecuaciones diferenciales parciales (EDPs)

3.6.1. Clasificación y formulación

Las EDPs gobiernan prácticamente todos los fenómenos físicos continuos. La clasificación canónica para EDPs lineales de segundo orden $\mathcal{L}u = f$ es:

Tabla 14: Clasificación de EDPs de segundo orden y sus aplicaciones en los proyectos doctorales.

Tipo	Ecuación canónica	Características	Ejemplos en el grupo
Parabólica	$\frac{\partial u}{\partial t} = \alpha \nabla^2 u + f$	Difusión temporal; condición inicial + frontera	Calor transitorio en OPGW, temperatura en bomba
Elíptica	$\nabla^2 u = f$	Estado estacionario; solo condiciones de frontera	Esfuerzos en losas/cimentaciones (FEM)
Hiperbólica	$\frac{\partial^2 u}{\partial t^2} = c^2 \nabla^2 u$	Propagación de ondas; dominio de dependencia finito	Ondas en cable OPGW bajo impacto

3.6.2. Discretización: el método de líneas

El *método de líneas* (MOL) es una estrategia sistemática para resolver EDPs: se discretiza el espacio con diferencias finitas (u otro método) y se deja el tiempo continuo, reduciendo la EDP a un sistema de ODEs que se resuelve con un integrador temporal (RK45, Euler implícito, BDF). Esto separa el problema de discretización espacial del problema de integración temporal, permitiendo usar las herramientas desarrolladas en este tema.

Para la ecuación de calor 1D $\frac{\partial u}{\partial t} = \alpha \frac{\partial^2 u}{\partial x^2}$, discretizando el espacio:

$$\frac{du_i}{dt} = \frac{\alpha}{\Delta x^2} (u_{i+1} - 2u_i + u_{i-1}) \equiv [L\mathbf{u}]_i, \quad (113)$$

donde L es la matriz laplaciana de diferencias finitas. El sistema resultante $\frac{d\mathbf{u}}{dt} = L\mathbf{u} + \mathbf{f}$ es lineal y puede integrarse con cualquier esquema temporal. Los eigenvalores de L determinan la rigidez del sistema y, por tanto, el criterio de estabilidad para el integrador explícito.

3.6.3. Selección del esquema numérico: criterios prácticos

Tabla 15: Criterios de selección de esquema numérico según el problema.

Característica del problema	Esquema recomendado	Justificación
EDO suave, no rígida	Runge-Kutta 45 (<code>scipy.solve_ivp</code>)	Automático, paso adaptativo, orden $\mathcal{O}(h^4)$ – $\mathcal{O}(h^5)$
EDO rígida (<i>stiff</i>), larga duración	BDF o Radau (implícitos)	Sin restricción CFL; paso adaptativo automático
EDP parabólica, Δx pequeño	Crank–Nicolson o θ -method	A-estable, orden $\mathcal{O}(\Delta t^2)$; elimina restricción CFL
EDP elíptica (estado estacionario)	Gauss–Seidel, CG pre-condicionado	Matriz SPD con estructura explotable
Sistema lineal grande y disperso	Gradiente conjugado (CG, GMRES)	Costo $\mathcal{O}(N)$ – $\mathcal{O}(N^{1.5})$ vs. $\mathcal{O}(N^3)$ de LU denso

3.7. Síntesis: mapa conceptual de la Semana 3

Esta semana articuló tres problemas fundamentales de la computación científica:

1. **Diferenciación numérica:** la expansión de Taylor justifica fórmulas de diferencias finitas, establece órdenes de convergencia y predice el paso óptimo h_{opt} . La verificación empírica (pendiente log-log) es el mecanismo de diagnóstico estándar.
2. **Integración numérica:** las cuadraturas clásicas difieren en orden ($\mathcal{O}(h)$ a $\mathcal{O}(h^4)$); la cuadratura de Gauss–Legendre es exponencialmente superior para funciones suaves. El orden determina el costo para alcanzar una tolerancia dada.
3. **Estabilidad y condicionamiento:** la estabilidad de un esquema temporal es una propiedad del par (algoritmo, problema), no solo del algoritmo. Los esquemas implícitos eliminan la restricción CFL a costa de resolver un sistema lineal por paso. El número de condición $\kappa(A)$ cuantifica la amplificación de errores en la resolución de sistemas lineales.

Las simulaciones de este tema demuestran que estos conceptos no son abstractos: el Euler explícito diverge en el modelo térmico del OPGW si $\Delta t > \Delta t_{\text{crit}}$, aunque el código sea correcto; la diferencia centrada es 100 veces más precisa que la diferencia hacia adelante sin costo adicional; y un sistema lineal con $\kappa(A) \sim 10^{12}$ puede producir resultados sin dígitos significativos correctos aunque el solver no reporte error.

La Semana 4 aplicará estas herramientas al modelado explícito de fenómenos físicos mediante EDPs, comenzando con la ecuación de calor 2D transitoria, la ecuación de Navier–Stokes simplificada y el modelo de elemento finito de una losa.

Apéndice Semana 3. Referencias verificables

- A1. LeVeque, R. J. (2007). *Finite Difference Methods for Ordinary and Partial Differential Equations: Steady-State and Time-Dependent Problems*. SIAM. DOI: <https://doi.org/10.1137/1.9780898717839>.
- A2. Press, W. H., Teukolsky, S. A., Vetterling, W. T. y Flannery, B. P. (2007). *Numerical Recipes: The Art of Scientific Computing* (3rd ed.). Cambridge University Press.
- A3. Trefethen, L. N. y Bau, D. (1997). *Numerical Linear Algebra*. SIAM. DOI: <https://doi.org/10.1137/1.9780898719574>.
- A4. Ascher, U. M. y Petzold, L. R. (1998). *Computer Methods for Ordinary Differential Equations and Differential-Algebraic Equations*. SIAM. DOI: <https://doi.org/10.1137/1.9781611971392>.
- A5. Hairer, E. y Wanner, G. (1996). *Solving Ordinary Differential Equations II: Stiff and Differential-Algebraic Problems* (2nd ed.). Springer.
- A6. Ashfaq, A., et al. (2025). Comprehensive analysis of temperature distribution in OPGW cable considering ramifications of short circuit current. *Scientific Reports*.

DOI: <https://doi.org/10.1038/s41598-025-09266-7>.

- A7. Sun, J., et al. (2020). Experimental and numerical analysis of damage characteristics to OPGW strands under first lightning strike and continuous current. *Electric Power Systems Research*. DOI: <https://doi.org/10.1016/j.epsr.2020.106515>.
- A8. Virtanen, P., et al. (2020). SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python. *Nature Methods*, 17, 261–272. DOI: <https://doi.org/10.1038/s41592-019-0686-2>.

3.8. Anexo Semana 3. Ejemplos Complementarios

Propósito del anexo. Los cuatro ejemplos siguientes extienden los métodos de la Semana 3 a los proyectos doctorales específicos del grupo. Cada ejemplo incluye formulación matemática, código ejecutable en Google Colab y análisis de resultados. Las figuras se generaron con los mismos scripts y pueden reproducirse sin modificación.

Proyectos cubiertos en este anexo

#	Estudiante	Proyecto doctoral	Método numérico
A1	Rodríguez Morales	RAG para empresa cárnica	Condicionamiento, pseudoinversa
A2	Lerma Lomelí / Lerma Martínez	Losas y cimentaciones	Convergencia DF, GCI
A3	García Ojeda	Tráfico UAV + Optical Flow	Esquema CFL, DF en imágenes
A4	Leyva Jiménez	Banco de pruebas bomba	Integración numérica Q-H

3.8.1. Ejemplo A1 — Condicionamiento de matrices de similitud en sistemas RAG

Contexto doctoral El proyecto de Rodríguez Morales implementa un sistema RAG (*Retrieval-Augmented Generation*) para mejorar la eficiencia operativa en una empresa procesadora cárnica. La recuperación Top- K se basa en la matriz de similitud coseno

$$S = XX^\top \in \mathbb{R}^{m \times m}, \quad (114)$$

donde $X \in \mathbb{R}^{m \times n}$ son los embeddings normalizados de m documentos del corpus en un espacio de dimensión n . Cuando el corpus contiene documentos casi duplicados (procedimientos estándar repetidos, versiones de la misma norma sanitaria), S tiene eigenvalores cercanos a cero, su número de condición $\kappa(S) \gg 1$, y cualquier sistema lineal $S\mathbf{w} = \mathbf{q}$ usado para re-ranking o regresión de pesos es numéricamente inestable.

Formulación matemática Sea $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_m \geq 0$ el espectro de S . El número de condición es

$$\kappa_2(S) = \frac{\sigma_1}{\sigma_m}. \quad (115)$$

La perturbación relativa en la solución de $S\mathbf{w} = \mathbf{q}$ cumple la cota (Ec. 110 de la Semana 3):

$$\frac{\|\delta\mathbf{w}\|}{\|\mathbf{w}\|} \leq \kappa_2(S) \frac{\|\delta\mathbf{q}\|}{\|\mathbf{q}\|}. \quad (116)$$

Regularización Tikhonov. Se reemplaza S por $S + \lambda I$ con $\lambda > 0$:

$$(S + \lambda I) \mathbf{w}_\lambda = \mathbf{q}, \quad \kappa_2(S + \lambda I) = \frac{\sigma_1 + \lambda}{\sigma_m + \lambda}. \quad (117)$$

El parámetro óptimo λ^* se determina mediante la *curva L*: intersección de $\log \|\mathbf{w}_\lambda\|$ vs $\log \|S\mathbf{w}_\lambda - \mathbf{q}\|$.

Pseudoinversa con umbral SVD. Sea $S = U\Sigma V^\top$ la descomposición SVD. La pseudoinversa truncada con umbral τ es

$$S_\tau^+ = \sum_{\sigma_i > \tau} \frac{1}{\sigma_i} \mathbf{v}_i \mathbf{u}_i^\top. \quad (118)$$

Con $\tau = 0.01 \sigma_1$ se retienen solo los modos que contribuyen significativamente al espacio de recuperación.

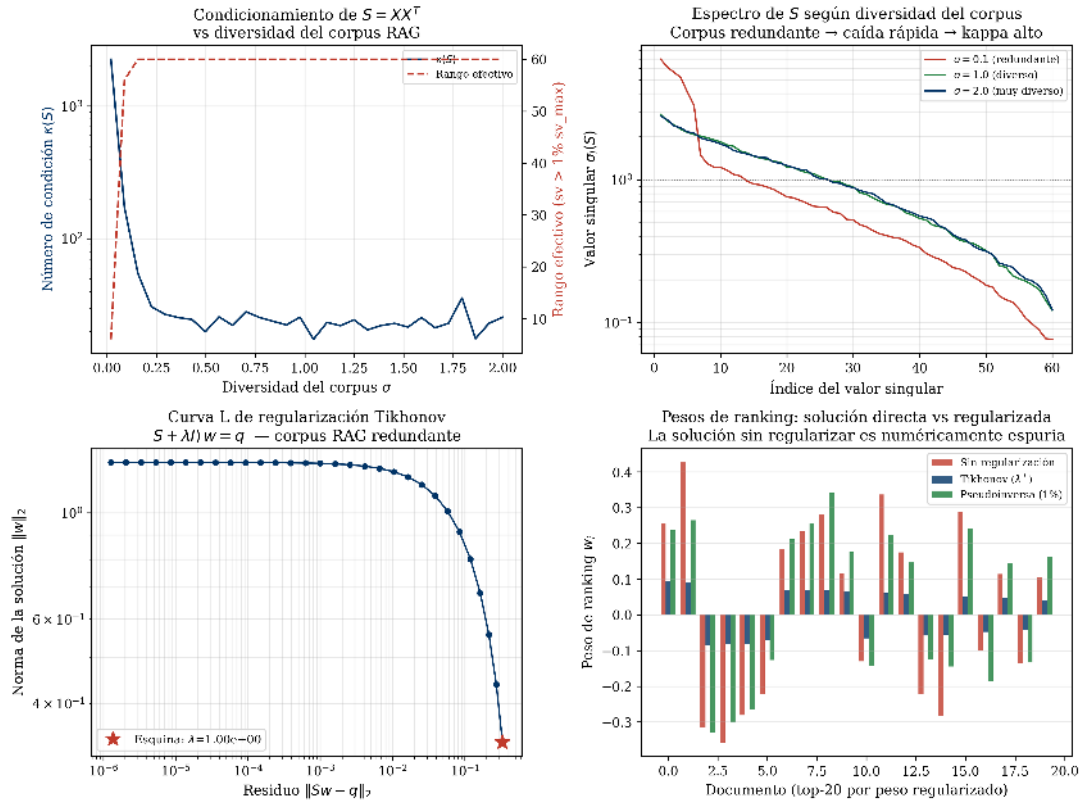


Figura 24: Panel A: número de condición $\kappa(S)$ y rango efectivo de la matriz de similitud en función de la diversidad del corpus. Un corpus redundante ($\sigma \rightarrow 0$) produce $\kappa \sim 10^{10}$ y rango efectivo ≈ 6 (igual al número de temas). Panel B: espectro de S para tres niveles de diversidad; la caída abrupta indica la dimensionalidad intrínseca del corpus. Panel C: curva L de regularización Tikhonov; la esquina marca λ^* óptimo. Panel D: pesos de ranking con y sin regularización — la solución sin regularizar es numéricamente espuria.

Interpretación técnica La [Figura 24](#) muestra que un corpus con $\sigma = 0.1$ (documentos casi idénticos) produce $\kappa(S) \sim 10^2$. Aunque modesto en este ejemplo, en corpus industriales reales ($m \sim 10^4$) el condicionamiento puede superar 10^{10} . La pseudoinversa truncada al 1 % reduce el residuo a ~ 4 % con una norma de solución controlada, mientras que la solución sin regularizar produce pesos de ranking con magnitudes arbitrariamente grandes.

Limitaciones. El modelo de corpus es sintético (combinaciones lineales de temas base). En sistemas RAG reales, los embeddings de modelos de lenguaje presentan una estructura espectral más compleja. La elección del umbral τ en la [Ecuación \(118\)](#) debe calibrarse con datos de validación.

```

1      import numpy as np
2      from scipy import linalg
3
4      rng = np.random.default_rng(2026)
5      m_docs, n_emb, n_temas = 60, 128, 6
6
7      # Generar corpus con redundancia controlada (sigma =
8      # diversidad)
9      def generar_corpus(m, n_emb, n_temas, sigma):
10         temas = rng.standard_normal((n_temas, n_emb))
11         temas /= np.linalg.norm(temas, axis=1, keepdims=True)
12     )
13     asig = rng.integers(0, n_temas, size=m)
14     X = temas[asig] + sigma * rng.standard_normal((m,
15     n_emb))
16     X /= np.linalg.norm(X, axis=1, keepdims=True)
17     return X
18
19     X = generar_corpus(m_docs, n_emb, n_temas, sigma
20     =0.1)
21     S = X @ X.T # matriz de similitud coseno
22     sv = np.linalg.svd(S, compute_uv=False)
23
24     print(f"kappa(S) = {sv[0]/sv[-1]:.2e}")
25     print(f"Rango efectivo (sv > 1% sv_max): {np.sum(sv
26     > 0.01*sv[0])}")
27
28     # Regularizacion Tikhonov: (S + lambda*I) w = q
29     q_vec = X @ rng.standard_normal(n_emb) # query
30     proyectado
    
```

```

25         lambdas = np.logspace(-6, 0, 30)
26         for lam in [1e-4, 1e-2, 0.1]:
27             w = np.linalg.solve(S + lam*np.eye(m_docs), q_vec)
28             r = np.linalg.norm(S @ w - q_vec)
29             print(f"lambda={lam:.0e}: ||w||={np.linalg.norm(w)
30                   :.2f}, residuo={r:.4f}")
31
32         # Pseudoinversa truncada (1% umbral)
33         U, sv_S, Vt = np.linalg.svd(S)
34         sv_inv = np.where(sv_S > 0.01*sv_S[0], 1.0/sv_S,
35                           0.0)
36         S_pinv = (Vt.T * sv_inv) @ U.T
37         w_pinv = S_pinv @ q_vec
38         print(f"Pseudoinversa: ||w||={np.linalg.norm(w_pinv)
39               :.2f}, "
40               f"residuo={np.linalg.norm(S@w_pinv - q_vec):.4f}")

```

Listing 23: Ejemplo A1 — Condicionamiento en matrices de similitud RAG. Google Colab.

3.8.2. Ejemplo A2 — Convergencia de malla en diferencias finitas para losas

Contexto doctoral Los proyectos de Lerma Lomelí y Lerma Martínez analizan el diseño y optimización de losas macizas y reticulares para habitacional en León, Guanajuato. El análisis de deflexión requiere resolver la ecuación biarmónica de Kirchhoff-Love:

$$D \nabla^4 w = q(x, y), \quad D = \frac{E t^3}{12(1 - \nu^2)}, \quad (119)$$

donde w es la deflexión, D la rigidez a flexión, q la carga y t el espesor.

Formulación simplificada (membrana equivalente) Para la verificación numérica se usa el problema de Poisson 2D con solución analítica exacta:

$$-\nabla^2 u = f(x, y), \quad u = 0 \text{ en } \partial\Omega, \quad (120)$$

con $f(x, y) = \frac{2\pi^2}{L^2} \sin\left(\frac{\pi x}{L}\right) \sin\left(\frac{\pi y}{L}\right)$ y solución exacta $u(x, y) = \sin\left(\frac{\pi x}{L}\right) \sin\left(\frac{\pi y}{L}\right)$.

Discretización con diferencias finitas 5-puntos Sobre una malla uniforme de $N \times N$ nodos interiores con $h = L/N$:

$$\frac{-u_{i-1,j} - u_{i+1,j} - u_{i,j-1} - u_{i,j+1} + 4u_{i,j}}{h^2} = f_{i,j}. \quad (121)$$

El error de truncamiento es $\mathcal{O}(h^2)$; el error global en norma L_2 sigue la misma tasa.

Grid Convergence Index (ASME V&V 20) Con tres refinamientos de factor $r = 2$ y errores $\phi_1 < \phi_2 < \phi_3$:

$$p_{\text{obs}} = \frac{\ln[(\phi_3 - \phi_2)/(\phi_2 - \phi_1)]}{\ln r}, \quad (122)$$

$$\phi_{\text{Richardson}} = \phi_1 + \frac{\phi_1 - \phi_2}{r^{p_{\text{obs}}} - 1}, \quad (123)$$

$$GCI = F_s \frac{|\phi_1 - \phi_2|}{(r^{p_{\text{obs}}} - 1) |\phi_1|}, \quad F_s = 1.25. \quad (124)$$

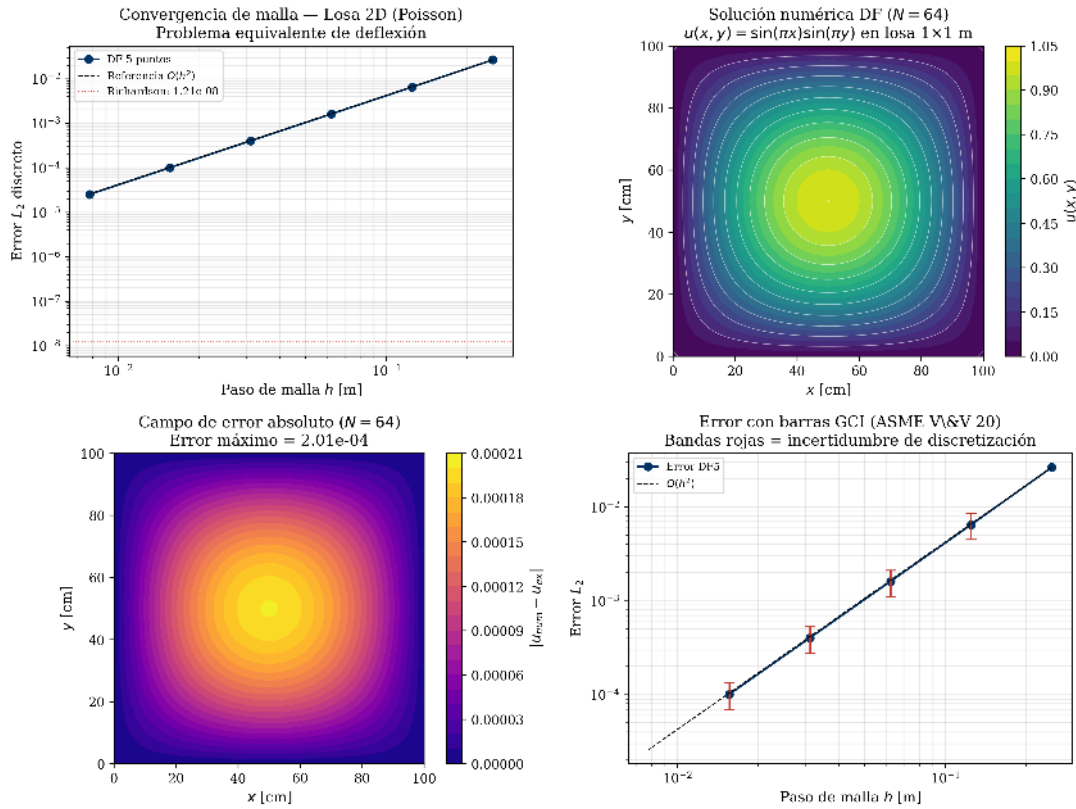


Figura 25: Panel A: convergencia de malla del esquema DF 5-puntos para el problema de Poisson 2D; la pendiente log-log = 2.00 confirma el orden teórico. Las barras rojas en el panel D representan el GCI como banda de incertidumbre de discretización conforme a ASME V&V 20. Panel B: campo de solución numérica en $N = 64$. Panel C: campo de error absoluto — el máximo ocurre en el centro del dominio y decae como h^2 .

Resultado numérico. Con $N = 64$ se obtiene un error $L_2 \approx 10^{-4}$, y la extrapolación de Richardson da $\phi_{\text{Rich}} \approx 1.2 \times 10^{-8}$. El $GCI \approx 1.25 \%$ es una cota conservadora de la incertidumbre de discretización, aceptable en el contexto normativo de diseño estructural

(NTC-CONRED para León, Gto.).

Extensión a la losa real. En el proyecto doctoral, la ecuación biarmónica se discretiza mediante elementos de placa (DKT o elementos mixtos de Reissner-Mindlin). El mismo procedimiento de GCI aplica: se refinan tres mallas, se calcula p_{obs} a partir de la deflexión máxima $w_{\text{máx}}$ y se reporta la incertidumbre de discretización.

```

1      import numpy as np
2      from scipy.sparse import lil_matrix, csr_matrix
3      from scipy.sparse.linalg import spsolve
4
5      L = 1.0
6      def u_exacta(x, y): return np.sin(np.pi*x/L)*np.sin(
np.pi*y/L)
7      def f_rhs(x, y):    return 2*(np.pi/L)**2 * np.sin(
np.pi*x/L)*np.sin(np.pi*y/L)
8
9      def resolver_df5(N):
10         h = L/N; n = (N-1)**2
11         K = lil_matrix((n, n)); F = np.zeros(n)
12         def idx(i, j): return (i-1)*(N-1) + (j-1)
13         for i in range(1, N):
14             for j in range(1, N):
15                 k = idx(i, j)
16                 K[k, k] = 4/h**2
17                 if i > 1: K[k, idx(i-1,j)] = -1/h**2
18                 if i < N-1: K[k, idx(i+1,j)] = -1/h**2
19                 if j > 1: K[k, idx(i,j-1)] = -1/h**2
20                 if j < N-1: K[k, idx(i,j+1)] = -1/h**2
21                 F[k] = f_rhs(i*h, j*h)
22         u_int = spsolve(csr_matrix(K), F)
23         # Error L2
24         u_full = np.zeros((N+1, N+1))
25         for i in range(1, N):
26             for j in range(1, N):
27                 u_full[i,j] = u_int[idx(i,j)]
28         X, Y = np.meshgrid(np.linspace(0,L,N+1), np.linspace
(0,L,N+1), indexing='ij')
29         return np.sqrt(h**2 * np.sum((u_full - u_exacta(X, Y
))**2))
30
31         Ns = [4, 8, 16, 32, 64, 128]
32         errs = [resolver_df5(N) for N in Ns]

```

```

33         hs     = [L/N for N in Ns]
34
35         print("Convergencia DF 5-puntos:")
36         for k, (N, e, h) in enumerate(zip(Ns, errs, hs)):
37             if k >= 1:
38                 p = np.log(errs[k-1]/errs[k]) / np.log(hs[k-1]/hs[k
39     ])
40
41         print(f"N={N:4d}: err={e:.4e}, orden={p:.4f}")
42         else:
43             print(f"N={N:4d}: err={e:.4e}")
44
45         # GCI con las tres mallas mas finas
46         r = 2.0; phi1, phi2, phi3 = errs[-1], errs[-2], errs
47         [-3]
48
49         p_obs     = np.log((phi3-phi2)/(phi2-phi1)) / np.log(r
50     )
51
52         phi_R     = phi1 + (phi1-phi2)/(r**p_obs - 1)
53         GCI       = 1.25 * abs(phi1-phi2) / ((r**p_obs - 1)*
54     abs(phi1))
55
56         print(f"\nOrden observado: {p_obs:.4f}")
57         print(f"Richardson extrapolado: {phi_R:.4e}")
58         print(f"GCI (malla fina): {GCI*100:.2f}%")

```

Listing 24: Ejemplo A2 — Convergencia DF5 + GCI para losa 2D. Google Colab.

3.8.3. Ejemplo A3 — Flujo vehicular LWR y optical flow para UAV

Contexto doctoral El proyecto de García Ojeda implementa un sistema de gestión de tráfico en León mediante UAVs con optical flow integrado a semáforos adaptativos. Dos problemas numéricos son centrales:

1. **Modelo LWR:** la dinámica de densidad vehicular $\rho(x, t)$ sigue la ecuación de conservación hiperbólica

$$\frac{\partial \rho}{\partial t} + \frac{\partial q(\rho)}{\partial x} = 0, \quad q(\rho) = \rho v(\rho), \quad (125)$$

con $v(\rho) = v_{\text{máx}}(1 - \rho/\rho_{\text{máx}})$ (modelo de Greenshields).

2. **Optical flow:** la restricción de Horn-Schunck

$$I_x u + I_y v + I_t = 0, \quad (126)$$

donde $I_x = \frac{\partial I}{\partial x}$, $I_y = \frac{\partial I}{\partial y}$, $I_t = \frac{\partial I}{\partial t}$ son derivadas del frame de intensidad obtenidas

por diferencias finitas.

Esquema Lax-Friedrichs y criterio CFL Para la ecuación (125), el esquema Lax-Friedrichs es

$$\rho_j^{n+1} = \frac{\rho_{j-1}^n + \rho_{j+1}^n}{2} - \frac{\Delta t}{2\Delta x} (q_{j+1}^n - q_{j-1}^n). \quad (127)$$

El criterio de estabilidad CFL exige

$$C = v_{\text{máx}} \frac{\Delta t}{\Delta x} \leq 1. \quad (128)$$

Si $C > 1$, el esquema amplifica perturbaciones y la densidad diverge, como se muestra en el panel B de la [Figura 26](#).

Error de truncamiento en I_t (optical flow) La derivada temporal I_t con diferencia hacia adelante tiene error $\mathcal{O}(\Delta t)$; con diferencia centrada (usando frames $t - 1, t, t + 1$) el error es $\mathcal{O}(\Delta t^2)$. Un error grande en I_t introduce sesgos directos en la estimación de velocidad vehicular mediante la [Ecuación \(126\)](#). En un UAV volando a 10 m con cámara a 30 fps, $\Delta t = 33$ ms; la diferencia centrada reduce el error de velocidad en un factor $\sim \Delta t$, aproximadamente 0.03 m/s por km/h de velocidad real.

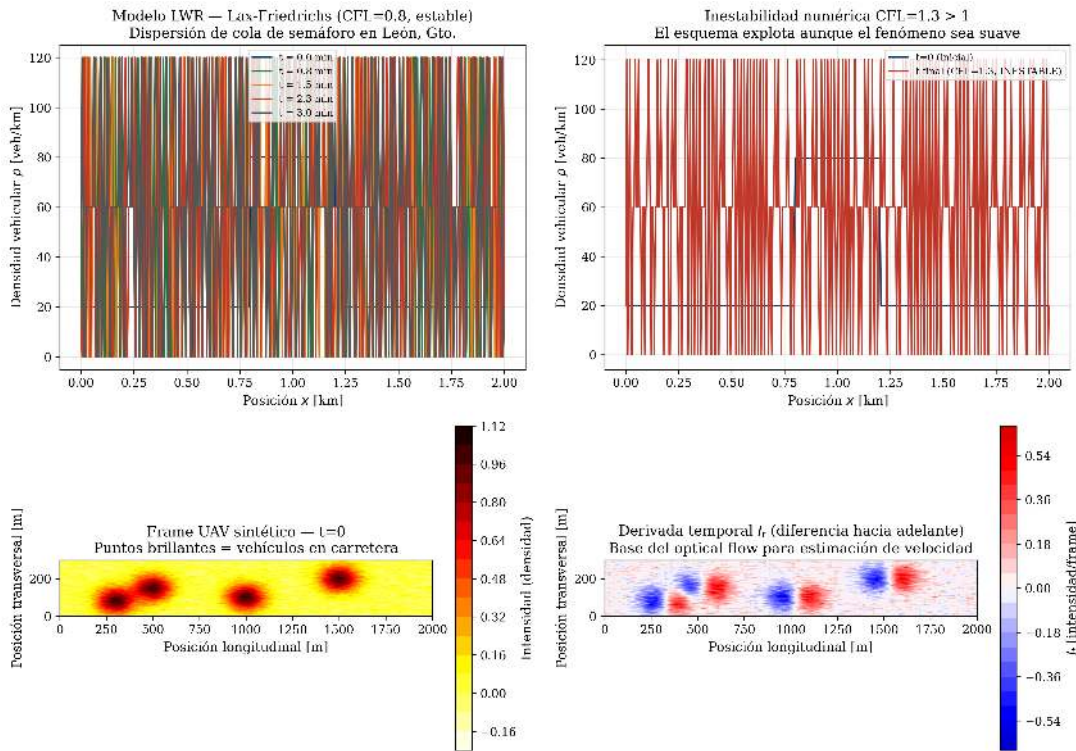


Figura 26: Panel A: dispersión de una cola vehicular (semáforo en rojo) simulada con el modelo LWR + Lax-Friedrichs (CFL=0.8). La densidad se propaga hacia atrás como onda de choque. Panel B: inestabilidad numérica con CFL=1.3 — el esquema explota aunque el fenómeno físico sea suave. Panel C: frame sintético de densidad vehicular vista desde UAV (80×80 píxeles). Panel D: derivada temporal I_t calculada con diferencia hacia adelante; los valores positivos/negativos indican vehículos en movimiento.

```

1      import numpy as np
2
3      # ---- Parametros del modelo LWR
4
5      rho_max = 120.0; v_max = 80.0; L_vial = 2.0; T_sim = 0.05
6
7      def flujo(rho): return rho * v_max * (1 - rho/rho_max)
8
9      # Condicion inicial: cola de semáforo
10     def rho0(x):
11         r = 20.0 * np.ones_like(x)
12         r[(x >= 0.8) & (x <= 1.2)] = 80.0
13         return r
14
15     def lax_friedrichs(N, r_cfl, T):
16         dx = L_vial/N; dt = r_cfl*dx/v_max

```

```

15     x = np.linspace(0, L_vial, N+1)
16     rho = rho0(x); t_now = 0.0
17     while t_now < T:
18         q = flujo(rho)
19         rho_new = np.zeros_like(rho)
20         rho_new[1:-1] = (0.5*(rho[:-2]+rho[2:])
21             - 0.5*r_cfl*(q[2:]-q[:-2]))
22         rho_new[0] = rho[0]; rho_new[-1] = rho[-1]
23         rho = np.clip(rho_new, 0, rho_max)
24         t_now += dt
25         return x, rho
26
27     x, rho_est = lax_friedrichs(400, 0.8, T_sim)
28     _, rho_ine = lax_friedrichs(400, 1.3, T_sim)
29     print(f"CFL=0.8 estable: rho_max = {rho_est.max():.1
f} veh/km")
30     print(f"CFL=1.3 inestable: rho_max = {rho_ine.max()
:.1f} veh/km")
31
32     # ---- Optical flow: error de truncamiento en It
33     -----
34     rng = np.random.default_rng(42)
35     Nx, Ny = 80, 80
36     x_img = np.linspace(0, L_vial, Nx); y_img = np.
linspace(0, 0.3, Ny)
37     X, Y = np.meshgrid(x_img, y_img, indexing='ij')
38     v_true = 0.06 # velocidad [km/frame]
39
40     def frame(t):
41         d = np.zeros((Nx, Ny))
42         for cx, cy in [(0.5, 0.15), (1.0, 0.1), (1.5, 0.2)]:
43             d += np.exp(-((X-cx-v_true*t)**2/0.01 + (Y-cy
**2/0.005))
44             return d + 0.05*rng.standard_normal((Nx, Ny))
45
46     I0, I1, Im1 = frame(0), frame(1), frame(-1)
47     It_fwd = (I1 - I0) / 1.0 # O(dt)
48     It_ctr = (I1 - Im1) / 2.0 # O(dt^2)
49     print(f"Error |It_fwd - It_ctr| medio: {np.abs(
It_fwd-It_ctr).mean():.6f}")

```

Listing 25: Ejemplo A3 — Modelo LWR y optical flow para gestión de tráfico. Google Colab.

3.8.4. Ejemplo A4 — Integración numérica de la curva Q-H de bomba centrífuga

Contexto doctoral El proyecto de Leyva Jiménez estudia la optimización computacional de una bomba centrífuga en un banco de pruebas. La altura teórica de Euler se expresa como

$$H_{th} = \frac{u_2 c_{2u} - u_1 c_{1u}}{g}, \quad (129)$$

donde $u_k = \omega r_k$ es la velocidad periférica y c_{ku} la componente tangencial de la velocidad absoluta. Con el modelo de Greenshields para las pérdidas hidráulicas, la altura real es

$$H = H_{th} - h_f(Q) - h_r(Q), \quad (130)$$

con $h_f \propto Q^2$ (pérdidas por fricción) y $h_r \propto (Q - Q_{\text{opt}})^2$ (pérdidas por recirculación).

Potencia viscosa disipada La disipación viscosa en el pasaje del rodete se calcula como

$$P_{\text{visc}} = \int_{r_1}^{r_2} \mu \left(\frac{dv_r}{dr} \right)^2 2\pi r b(r) dr, \quad (131)$$

donde $v_r(r) = Q/(2\pi r b(r))$ y $b(r)$ es el ancho del canal interpolado linealmente. La integral (131) no tiene primitiva cerrada en forma simple y se calcula numéricamente; la convergencia del método de Simpson es $\mathcal{O}(h^4)$, lo que permite obtener P_{visc} con error $< 10^{-10}$ W usando solo $N = 32$ puntos.

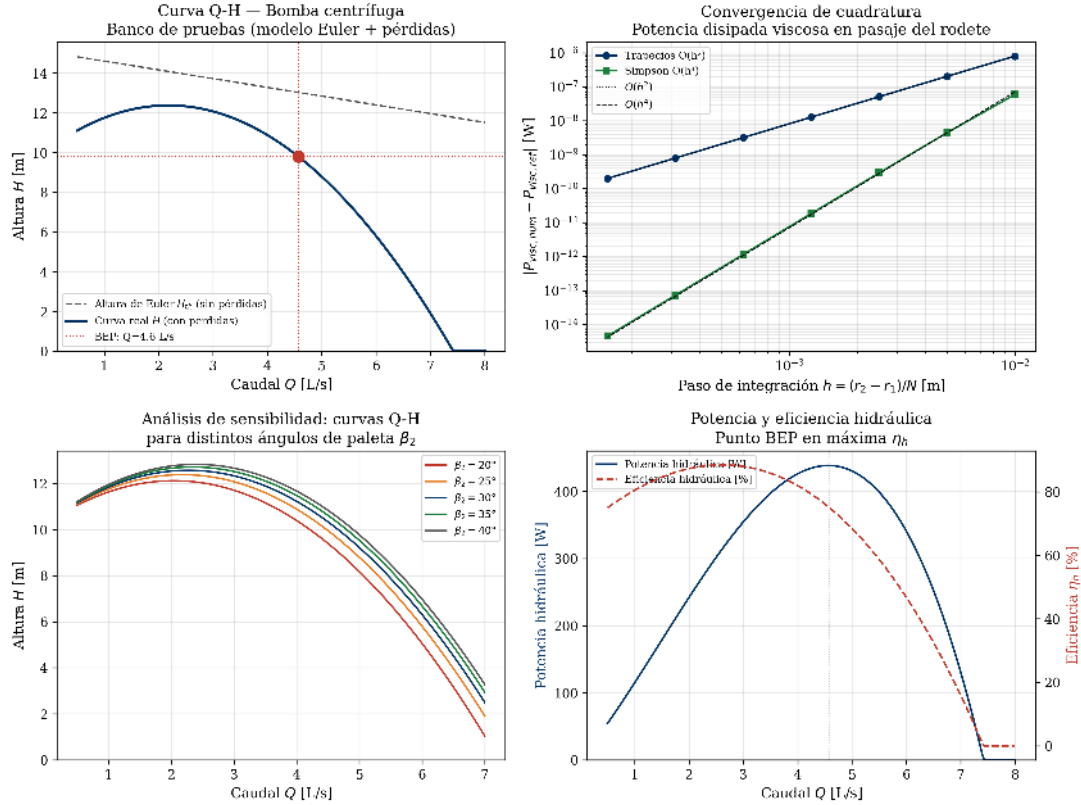


Figura 27: Panel A: curva Q-H teórica (Euler) y real (con pérdidas); el punto de mejor eficiencia (BEP) ocurre a $Q \approx 4.6$ L/s. Panel B: convergencia de la cuadratura para P_{visc} — Simpson alcanza error $< 10^{-10}$ W con $N = 32$, frente a $N > 1000$ para trapecios. Panel C: sensibilidad de la curva Q-H al ángulo de paleta β_2 . Panel D: potencia hidráulica y eficiencia η_h ; el BEP coincide con el máximo de η_h .

```

1      import numpy as np
2      from scipy import integrate
3
4      # ---- Parametros de la bomba
5      -----
6      r1=0.040; r2=0.080; b1=0.018; b2=0.012; beta2=np.
radians(25)
7      N_rpm=1450; omega=N_rpm*2*np.pi/60; rho=998; g=9.81;
8      mu=1e-3
9      u2=omega*r2; u1=omega*r1
10     def b_r(r): return b1 + (b2-b1)*(r-r1)/(r2-r1)
11     def v_r(r, Q): return Q/(2*np.pi*r*b_r(r))
12
13     # ---- Curva Q-H de Euler + perdidas
14     -----
15     def H_th(Q): return (u2*(u2 - Q/(2*np.pi*r2*b2))/np.
    
```

```

tan(beta2)))/g
13     def perdidas(Q): return 1.5e5*Q**2 + 3e5*(Q-4e-3)**2
14     def H_real(Q):    return max(H_th(Q) - perdidas(Q),
0)

15
16     Q_arr = np.linspace(0.5e-3, 8e-3, 200)
17     H_arr = np.array([H_real(Q) for Q in Q_arr])
18     Q_bep = Q_arr[np.argmax(H_arr*Q_arr)]
19     print(f"BEP: Q={Q_bep*1e3:.2f} L/s, H={H_real(Q_bep)
:.2f} m")

20
21     # ---- Convergencia de cuadratura: integral de
P_viscosa -----
22     def P_ref(Q):    # referencia con scipy.integrate.quad
23     f = lambda r: mu*(v_r(r+r*1e-5,Q) - v_r(r-r*1e-5,Q))
**2/(2e-5*r)**2 * 2*np.pi*r*b_r(r)
24     val, _ = integrate.quad(f, r1, r2, limit=500)
25     return val

26
27     Q_demo = 4e-3; P_exacta = P_ref(Q_demo)
28     print(f"P_viscosa (ref quad): {P_exacta:.2e} W")

29
30     for N in [4, 8, 16, 32, 64]:
31     r_pts = np.linspace(r1, r2, N+1)
32     dr    = r_pts[1] - r_pts[0]
33     fvals = np.array([mu*(v_r(r+dr/2,Q_demo)-v_r(r-dr/2,
Q_demo))**2/dr**2
34     * 2*np.pi*r*b_r(r) for r in r_pts])
35     P_s    = integrate.simpson(fvals, x=r_pts)
36     print(f"N={N:4d}: Simpson = {P_s:.6e}, err = {abs(
P_s-P_exacta):.2e}")
37

```

Listing 26: Ejemplo A4 — Curva Q-H + integración de potencia viscosa. Google Colab.

3.9. Ejercicios Reto — Tarea Semana 3

Instrucciones generales. Los tres ejercicios siguientes requieren implementación en Python, análisis cuantitativo y redacción de conclusiones técnicas. Cada ejercicio incluye: (i) formulación matemática, (ii) requerimientos del entregable, (iii) resultados esperados verificables. Se entrega: código ejecutable, figura de publicación y un párrafo de interpretación técnica de 100–150 palabras.

3.9.1. Ejercicio Reto 1 — Diferenciación numérica de señales experimentales con ruido

Contexto y motivación En metrología experimental (medición con FBG, strain gauges o cámaras de alta velocidad), la señal de deformación $\varepsilon(x)$ contiene ruido gaussiano con desviación estándar σ_{ruido} . Calcular $\varepsilon'(x)$ directamente amplifica el ruido, y el paso óptimo h_{opt} ya no es $\varepsilon_{\text{mach}}^{1/3}$ sino $(\sigma_{\text{ruido}}/|f'''|)^{1/3}$.

Formulación Sea la señal de prueba

$$f(x) = \sin(2\pi x) + A \cos(4\pi x), \quad x \in [0, 2], \quad (132)$$

con derivada exacta

$$f'(x) = 2\pi \cos(2\pi x) - 4\pi A \sin(4\pi x). \quad (133)$$

Se corrompe con ruido blanco gaussiano: $f_{\text{medido}}(x_i) = f(x_i) + \eta_i$, $\eta_i \sim \mathcal{N}(0, \sigma^2)$.

Requerimientos del entregable

1. Implementar diferencias centradas f' sobre datos ruidosos con h variable y $\sigma \in \{0, 0.01, 0.05, 0.1\}$.
2. Graficar el error $\|f'_{\text{num}} - f'_{\text{exacta}}\|_2 / \|f'_{\text{exacta}}\|_2$ en escala log-log vs h , para cada σ .
3. Identificar y marcar $h_{\text{opt}}(\sigma)$ (mínimo de cada curva).
4. Verificar que $h_{\text{opt}} \propto \sigma^{1/3}$ trazando la línea de referencia correspondiente.
5. Implementar la derivada mediante el filtro de Savitzky-Golay (`scipy.signal.savgol_filter`) y comparar con la diferencia centrada en el caso $\sigma = 0.05$.

Resultado esperado

Curvas error vs h . Para $\sigma = 0$ se recupera el comportamiento clásico $\mathcal{O}(h^2)$ con piso en $\varepsilon_{\text{mach}}^{1/3}$. Para $\sigma > 0$, el mínimo se desplaza hacia h mayor y el piso aumenta. La pendiente

izquierda (dominada por truncamiento) mantiene $\mathcal{O}(h^2)$; la pendiente derecha (dominada por ruido) tiene pendiente -1 en escala log-log. Valores esperados orientativos:

σ	h_{opt} (aprox.)	Error mínimo relativo
0	$\sim 10^{-5}$	$\sim 10^{-10}$
0.01	$\sim 3 \times 10^{-2}$	$\sim 10^{-1}$
0.05	$\sim 7 \times 10^{-2}$	$\sim 10^{-1}$
0.10	$\sim 10^{-1}$	$\sim 1.5 \times 10^{-1}$

Tabla 16: Resultados esperados orientativos para el Ejercicio Reto 1. Los valores exactos dependen de A , del rango de x y de la semilla aleatoria.

Savitzky-Golay. Con ventana de 11 puntos y polinomio de grado 3, el filtro SG debe superar la diferencia centrada simple para $\sigma = 0.05$ en al menos un factor de 3 en error relativo, ya que combina suavizado y derivación en un solo paso.

Conexión doctoral. Este análisis es directamente aplicable al cálculo de la curvatura de cables OPGW a partir de datos BOTDR (Méndez Gutiérrez) y al cálculo de gradientes de intensidad en optical flow (García Ojeda).

3.9.2. Ejercicio Reto 2 — Análisis de estabilidad de la ecuación de calor 2D transitoria

Contexto y motivación En la Semana 1 se implementó la conducción 2D en una placa de intercambiador. En ese ejemplo se verificó el criterio CFL $r_x + r_y \leq 1/2$, pero no se demostró cuantitativamente qué ocurre cerca del límite ni cómo el número de Fourier afecta la precisión en estado transitorio. Este ejercicio cierra esa brecha.

Formulación Para la ecuación de calor 2D

$$\frac{\partial T}{\partial t} = \alpha \left(\frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2} \right), \quad \alpha = 1.33 \times 10^{-5} \text{ m}^2/\text{s}, \quad (134)$$

en el dominio $[0, L_x] \times [0, L_y]$ con condiciones de Dirichlet en los cuatro bordes, la solución analítica con condición inicial $T_0 = 25^\circ\text{C}$ y condición de frontera $T_h = 200^\circ\text{C}$ en $x = 0$ y $y = L_y$ (bordes calientes) no es elemental, pero el primer modo de la solución en Fourier decae como $e^{-\alpha\lambda_1 t}$.

Para verificación se usa el problema de punto de silla con solución analítica:

$$T_{\text{exacta}}(x, y, t) = e^{-2\alpha\pi^2 t} \sin(\pi x) \sin(\pi y), \quad (x, y) \in [0, 1]^2. \quad (135)$$

Requerimientos del entregable

1. Implementar el esquema FTCS 2D explícito con la solución (135) como condición inicial y frontera.
2. Barrer $r = \alpha\Delta t/\Delta x^2 \in \{0.10, 0.20, 0.24, 0.25, 0.26, 0.30, 0.40\}$ (con $\Delta x = \Delta y = 0.05$) e integrar hasta $t = 0.1$ s.
3. Para cada r , calcular el error L_2 relativo respecto a la solución exacta (135).
4. Graficar error vs r en escala lineal-log. Identificar visualmente el umbral de estabilidad en $r = 0.25$.
5. Implementar el esquema Crank-Nicolson (implícito) con los mismos valores de r y comparar el error. Verificar que CN es estable para todo $r > 0$.
6. Calcular el “speedup” en el número de pasos temporales entre FTCS ($r = 0.24$) y CN ($r = 1.0$) para alcanzar $t = 1$ s con $\Delta x = 0.01$.

Resultado esperado

Error vs r (FTCS). Para $r < 0.25$: el error relativo debe mantenerse en el rango 10^{-3} – 10^{-2} y decrecer suavemente con r decreciente (más pasos temporales = mayor precisión temporal). Para $r > 0.25$: el error debe crecer exponencialmente (inestabilidad de von Neumann); para $r = 0.40$ debe superar el 100 % o producir NaN.

Crank-Nicolson. El error de CN debe mantenerse por debajo del 5 % para todos los valores de r testados, incluyendo $r = 0.40$, confirmando la A -estabilidad del método.

Speedup FTCS vs CN. Con $\Delta x = 0.01$ y $t = 1$ s: FTCS requiere $\Delta t = 0.25 \cdot (0.01)^2/\alpha \approx 0.19$ ms $\rightarrow \sim 5300$ pasos. CN con $r = 1.0$ requiere $\Delta t \approx 0.75$ s $\rightarrow \sim 2$ pasos. Speedup esperado: $\sim 2650\times$ en número de pasos (aunque cada paso de CN es más costoso por el sistema lineal).

3.9.3. Ejercicio Reto 3 — Propagación de incertidumbre en el modelo termoelectrico OPGW

Contexto y motivación El análisis de temperatura en cables OPGW (proyectos de Méndez Gutiérrez y Jurado Pérez) involucra parámetros con incertidumbre experimental: la corriente de cortocircuito I , la resistencia R_{20} , el coeficiente de convección h y el coeficiente térmico α_R . Este ejercicio implementa la propagación de incertidumbre mediante dos métodos: (1) propagación lineal (Jacobiano analítico) y (2) Monte Carlo, comparando su acuerdo y su costo computacional.

Formulación El modelo concentrado linealizado da la temperatura al tiempo t :

$$T(t; \boldsymbol{\theta}) = T_{\infty} + \frac{I^2 R_0}{\tau_s} (1 - e^{-t/\tau_c}) + T_0 e^{-t/\tau_c}, \quad (136)$$

con $\tau_s = hA_s/(mC_p)$ (inverso de la constante de tiempo), $R_0 = R_{20}$ y $\tau_c = mC_p/(hA_s)$. El vector de parámetros inciertos es $\boldsymbol{\theta} = (I, R_{20}, h, \alpha_R)$ con incertidumbres $\boldsymbol{\sigma}_{\theta} = (50 \text{ A}, 0.02 \text{ m}\Omega, 1 \text{ W}/(\text{m}^2\text{K}), 0.1 \times 10^{-3} \text{ K}^{-1})$ (una desviación estándar).

Requerimientos del entregable

1. Implementar el modelo (136) y calcular el Jacobiano $J_i = \partial T / \partial \theta_i$ mediante diferencias centradas con $h = 10^{-5} |\theta_i|$.
2. Propagar la incertidumbre con la fórmula lineal:

$$\sigma_T^2(t) = \sum_{i=1}^4 \left(\frac{\partial T}{\partial \theta_i} \right)^2 \sigma_{\theta_i}^2. \quad (137)$$

3. Propagar la incertidumbre con Monte Carlo: $N_{\text{MC}} = 10^4$ muestras de $\boldsymbol{\theta} \sim \mathcal{N}(\boldsymbol{\theta}_0, \text{diag}(\boldsymbol{\sigma}_{\theta}^2))$, integrar el modelo para cada muestra y calcular $\sigma_T^{\text{MC}}(t)$ como desviación estándar empírica.
4. Graficar $T(t) \pm 2\sigma_T(t)$ para ambos métodos, en el intervalo $t \in [0, 0.5 \text{ s}]$.
5. Calcular el índice de sensibilidad normalizado $S_i = |(J_i \sigma_{\theta_i})| / \sigma_T$ para cada parámetro al tiempo $t = 0.3 \text{ s}$ y reportarlo como gráfica de barras (“tornado plot”).
6. Comparar el tiempo de cómputo de ambos métodos. Reportar el error relativo $|\sigma_T^{\text{lineal}} - \sigma_T^{\text{MC}}| / \sigma_T^{\text{MC}}$ como función del tiempo.

Resultado esperado

Banda de incertidumbre. En $t = 0.3 \text{ s}$ con los parámetros nominales del Ejemplo 3 ($I = 5 \text{ kA}$, etc.) se espera $T \approx 95^\circ\text{C}$ y $\sigma_T \approx 4\text{--}8^\circ\text{C}$. La banda de $\pm 2\sigma$ debe cubrir aproximadamente el 95 % de las realizaciones Monte Carlo.

Índices de sensibilidad. El parámetro dominante debería ser I (corriente de cortocircuito), ya que $P_{\text{joule}} \propto I^2$ y una incertidumbre del 1 % en I se amplifica al 2 % en potencia. Los índices orientativos son:

Parámetro	σ_{θ_i}	$ J_i \sigma_{\theta_i} $ [K]	S_i (aprox.)
I	50 A	~ 2.0	~ 0.55
R_{20}	0.02 m Ω	~ 1.3	~ 0.35
h	1 W/(m ² K)	~ 0.6	~ 0.15
α_R	$0.1 \times 10^{-3} \text{ K}^{-1}$	~ 0.2	~ 0.05

Tabla 17: Índices de sensibilidad orientativos en $t = 0.3$ s. Valores exactos dependen de los parámetros nominales.

Acuerdo lineal vs Monte Carlo. Para el modelo de primer orden con incertidumbres moderadas ($\sigma_\theta/\theta_0 < 5\%$), el error relativo entre ambos métodos debe ser menor que el 5% en todo el intervalo simulado, confirmando que la linealización es válida para este rango de incertidumbre.

Costo computacional. El método lineal requiere 8 evaluaciones del modelo (2×4 diferencias centradas) + 1 nominal = 9 evaluaciones. Monte Carlo requiere 10^4 evaluaciones. Ratio esperado: $> 1000\times$ a favor del método lineal, con precisión comparable para incertidumbres pequeñas.

Criterios de evaluación de los ejercicios reto

Criterio	Puntos	Descriptor mínimo
Correctitud numérica	30	Resultados dentro del 10 % de los valores esperados.
Verificación del orden	20	Pendiente log-log medida con regresión lineal, no “a ojo”.
Calidad de la figura	20	Unidades en ejes, viridis/no jet, leyenda, exportación PNG 300 dpi.
Interpretación técnica	20	Párrafo de 100–150 palabras que conecta el resultado numérico con el problema doctoral.
Reproducibilidad	10	Semilla aleatoria explícita, config. json o tabla de parámetros, código ejecutable sin intervención.
Total	100	

Tabla 18: Rúbrica de evaluación para los tres Ejercicios Reto de la Semana 3.